

DECENTRALIZED PROCESSING FOR DISTANCE BASED FORMATION CONTROL

A Thesis

Submitted to the Graduate Faculty of the
Louisiana State University and
Agricultural and Mechanical College
in partial fulfillment of the
requirements for the degree of
Master of Science in Mechanical Engineering

in

The Department of Mechanical Engineering

by

Tonmoy Sarker

B.Sc. in Aeronautical Engineering, Military Institute of Science and Technology,
Dhaka, Bangladesh, 2017.

August 2021

Acknowledgements

I am grateful to Dr. Marcio de Queiroz for his continuous guidance and support throughout this project. I would like to express my gratitude to Dr. Guoxiang Gu and Dr. Corina Barbalata for agreeing to serve as my thesis committee. I would also like to thank Dr. Hunter B. Gilbert for his selfless support in various aspects of this project.

I thank all my current and ex lab colleagues in the iCORE lab for all the good memories we shared together. Finally, I give my deepest gratitude to my family and friends, whose love and constant support has been my driving force towards the completion of this work.

Table of Contents

Acknowledgements	ii
List of Tables	iv
List of Figures	v
Abstract	vi
Chapter 1. Introduction	1
1.1. Motivation	1
1.2. Related Work.....	3
1.3. Research Objective and Thesis Organization.....	4
Chapter 2. TIGER Square Testbed	6
2.1. Testbed Arena	6
2.2. Control Station	6
2.3. TIGERBots.....	8
2.4. Testbed Performance Synopsis	11
2.5. Operating Modes	12
Chapter 3. Decentralized Processing and Communication.....	15
3.1. Communication Protocols	15
3.2. Decentralized Processing Firmware.....	20
Chapter 4. Formation Control Experiments.....	24
4.1. Graph Theory and Formation Control Problems.....	24
4.2. Experimental Results.....	30
Chapter 5. Conclusions and Future Work.....	44
5.1. Conclusions	44
5.2. Future Work	45
References	48
Vita.....	50

List of Tables

Table 2.1. Major performance parameters of the TIGER Square subsystems.....	11
Table 2.2. TIGER Square Recommended Step-time for Different Number of Robots.....	12
Table 4.1. List of Formation Acquisition Experiments.	31
Table 4.2. Summary of Formation Acquisition Experiments.	36
Table 4.3. List of Formation Maneuvering Experiments.....	37
Table 4.4. Summary of Formation Maneuvering Experiments.	42

List of Figures

Figure 2.1. TIGER Square test bed and its subsystems..	7
Figure 2.2. Robot localization in the global frame using the control station's image extraction function.	8
Figure 2.3. TIGERBot fleet. Individual robots with and without the IR sensor board.....	9
Figure 2.4. TIGERBot main board top view and bottom view.....	9
Figure 2.5. TIGERBot IR sensor board components.....	10
Figure 2.6. TIGER Square Operating Modes layout..	13
Figure 3.1. Centralized communication working flowchart..	17
Figure 3.2. Decentralized communication working flowchart.	18
Figure 3.3. TIGERBot Firmware simplified workflow in <i>Modes 1, 2 and 3</i> combined.....	22
Figure 4.1. Directed and undirected graphs between 2 agents and 3 agents, respectively.	25
Figure 4.2. Robotic vehicle model with unicycle dynamics.....	27
Figure 4.3. Framework ambiguity examples.	29
Figure 4.4. Formation acquisition with three robots.....	32
Figure 4.5. Formation acquisition with four robots.	34
Figure 4.6. Formation acquisition with six robots.	35
Figure 4.7. Formation maneuvering with three robots.	38
Figure 4.8. Formation maneuvering with four robots.....	39
Figure 4.9. Formation maneuvering with six robots.....	40
Figure 4.10. Formation maneuvering with nine robots.....	41

Abstract

The indoor multi-agent robotic testbed named *TIGER Square* at the LSU iCORE lab is aimed at experimentally validating various formation control laws. The testbed is currently being refined to implement distance-based formation control algorithms in *decentralized* control scheme in terms of effective sensing of the robot localization, communication protocols, and the control algorithm processing. The scope of work for this research is to modify the testbed subsystems for decentralized processing and establish that the modifications do not affect the formation performance when compared to the fully centralized mode of operation. The decentralized processing of the control algorithms is executed by robots' upgraded firmware that is introduced in this work. The modifications were validated through a few formation acquisition and formation maneuvering experiments under different modes of operation and varying number of robots in the formation. Preliminary results towards fully decentralized communication between the robots were also presented in this study. The results obtained in this study would serve as a stepping-stone for successfully operating the system in fully decentralized manner. Moreover, it would allow us to consider practical scenarios where a need may arise to switch from centralized mode to decentralized mode due to, for example, loss of GPS or communication with the control station.

Chapter 1. Introduction

1.1. Motivation

Collective behavior exhibited by large groups of biological entities is a prevalent phenomenon in nature. At the microscopic level, we find such behaviors in bacterial gliding or cancer cell metastasis. Other examples include ‘living’ structures built by ant colonies to move through large gaps and high-speed maneuvering of large bird flocks. Collaboratively, these organisms can perform highly complex tasks even in unknown environments, which would otherwise be unattainable by a single organism. The study of these cohesive behaviors in the context of *engineered* systems is often called multi-agent systems, swarming, networked systems, or cooperative control.

A multi-agent system can be defined as a coordinated group of physical agents operating in a shared environment, tasked with the execution of a collective conflicting goal. Together the agents offer more scalability, robustness, and adaptability over individual agents during execution of complex tasks. Over time, research interests in autonomous technologies have been shifting from stand-alone platforms towards multi-agent robotic platforms or sensor networks. Multi-agent systems could offer a wide range of potential applications including surveillance in large areas, cooperative transportation of large objects, disaster response, and space exploration. Even in our day-to-day life, we have been experiencing a high volume of their involvement from self-organizing warehouse robots to self-driving cars or delivery drones.

The challenges with multi-agent systems come from the complexities of agent coordination and cooperation as it involves incorporating effective sensing methods, communication protocols, and collision avoidance algorithms. For multi-agent coordination, a control scheme can be *centralized* or *decentralized*. In the former, agent localization, computation of control laws, and

communication are executed by a central entity. For example, a central computer could use a Global Positioning System (GPS) to get agent localization data, compute the control algorithm, and then send control commands to the agents. The decentralized scheme, on the other hand, requires the agents to localize themselves and execute the control laws using onboard hardware and software components. In practice, one single mode might not provide the best solution and performance; e.g., loss of the GPS signal would require agents to switch to a decentralized scheme. Therefore, to improve robustness of the system, agents should be able to seamlessly switch between centralized and decentralized schemes.

Formation control is an important problem in multi-agent system coordination where the objective is for agents to form a prescribed geometric shape in space. A well-known method for solving this problem involves regulating a set of inter-agent distances to values prescribed by the desired shape. This method is commonly referred to as *distance-based* formation control [16],[23]. The main advantage of distance-based formation control is that the control algorithms are implementable in a fully decentralized manner.

Customizable proof-of-concept multi-agent testbeds have been gaining popularity among various research groups, as they offer the opportunity to experimentally validate state-of-the-art control algorithms before implementing them in real-world systems. An indoor multi-agent testbed called *TIGER Square* has been developed at the LSU iCORE Lab with the goal of implementing various formation control laws and analyzing their behavior under various constraints. In version I of TIGER Square, the testbed was only operable in a fully centralized manner. Specifically, a laptop computer processed all control algorithms and wirelessly transmitted the input commands to each robotic vehicle while an overhead camera, akin to an “indoor GPS”, provided position feedback. The purpose of this research project was to contribute to version II of

Tiger Square by making the appropriate hardware and software modifications that allow the testbed to be run in decentralized mode. Specifically, our goal was to decentralize the *control algorithm processing* component of the testbed.¹

1.2. Related Work

With the growth of research in multi-agent system coordination, many experimental robotic platforms have been developed throughout the world. Factors such as scalability, development time, cost, and ease of use and maintenance play pivotal roles in building such testbeds. A number of such multi-agent testbeds and their technical features can be found in [1],[24]. Among them the Robotarium testbed and their robots called GRITSBot [3] specifically stand out as it is fully open-source and remotely accessible. In fact, many of the centralized control features of TIGER Square were inspired by the Robotarium.

Most of these indoor testbeds feature a global localization system with an overhead camera or array of cameras. However, to achieve decentralized sensing capability, each robot must be equipped with onboard localization hardware that can measure range and bearing to the neighboring agents. In the literature, a handful of methods have been used to accomplish decentralized localization, each having pros and cons. In this regard, computer-vision techniques turn out to be the most reliable. However, they are costly and not practical for small-scale robots. Many commercial robotic platforms utilize them for high-end applications [4]. A lower-cost option is using Ultrasonic sensors, which calculate time-of-flight of the signal for range calculation [7,9,15]. However, they have slow refresh rates, and they are not much reliable in enclosed areas because of reflections [15].

¹ Sensing decentralization for Tiger Square was recently developed in [19].

Some other platforms use ultra-wide band radio technology with overall better performance [2]. This option is better suited for long-range applications because of the large antennas required. A frequently used method for small-scale localization is infrared (IR) detection using an array of photodiodes and LEDs [6,11-13]. IR hardware is lighter in weight and has omnidirectional localization capabilities. TIGER Square adopted the same IR technique for decentralized sensing [19] based on a method provided in [13] for converting IR sensor signals into range and bearing measurements.

A few projects featuring IR localization modules use direct IR communication between agents for sending sensing information [6,18], while others utilize Bluetooth and Wi-Fi modules and Internet-of-Things techniques for distribution of sensor data [9,10]. TIGER Square currently depends on communication with the central computer for sensor data distribution [19]. However, the robot hardware is capable of wireless data transfer between them.

1.3. Research Objective and Thesis Organization

The scope of work for this research was to refine the TIGER Square subsystems for decentralized processing of the formation control algorithms being developed at iCORE. We also presented preliminary results towards decentralized communication between the robots. The modifications made to TIGER Square are validated by comparing the performance of the formation control algorithms under the different modes of operation. Experimental runs are also conducted with varying number of robots to make the sure that the modifications are robust to the number of robots in the formation. We seek to establish that the hardware and software modifications made for decentralization do not affect the testbed formation performance when compared to the fully centralized mode of operation (benchmark).

The reminder of this thesis is organized as follows. Chapter 2 begins with an overview of the TIGER Square testbed. The chapter also introduces the four operating modes for the testbed based on sensing method, communication, and processing of the formation control algorithms. Chapter 3 describes the communication protocols used in TIGER Square for distribution of data and coordination of the robots during experiments. Then, the modifications to the TIGERBot firmware for decentralized algorithm processing are discussed. Chapter 4 outlines the formation control algorithms that were used in our comparative study and presents and discusses the experimental results. Finally, Chapter 5 contains the conclusions and recommendations for future work.

A portion of this work will be presented as a poster in the 2021 IEEE/ASME International Conference on Advanced Intelligent Mechatronics [21].

Chapter 2. TIGER Square Testbed

This chapter gives an overview of the TIGER Square testbed and its subsystems. TIGER Square testbed has three major components: the testbed arena, the control station, and the mobile robots, called TIGERBots. A more detailed description of TIGER Square can be found in [22].

2.1. Testbed Arena

The arena is an enclosed square platform of size 1.5 x 1.5 m. The platform is made of plywood with epoxy-coated non-slip surface for smooth movement of the robot wheels. The platform is supported by a steel frame underneath with four small feet in the corners for level adjusting (Figure 2.1).

A reverse U-shaped telescopic PVC structure attached to the platform supports two overhead camera housings (one for robot localization and one for recording videos of the experiments) and one directional patch antenna. The height of the PVC structure can be adjusted vertically to achieve the desired field of view from the cameras.

2.2. Control Station

TIGER Square's control station includes the control computer, an ethernet-connected Wi-Fi router, and the overhead USB cameras for localization and video recording. MATLAB is used as the control software. It initializes/interrupts all experimental the trials via wireless communication with the robots. It also extracts data from the overhead cameras, processes and stores them.

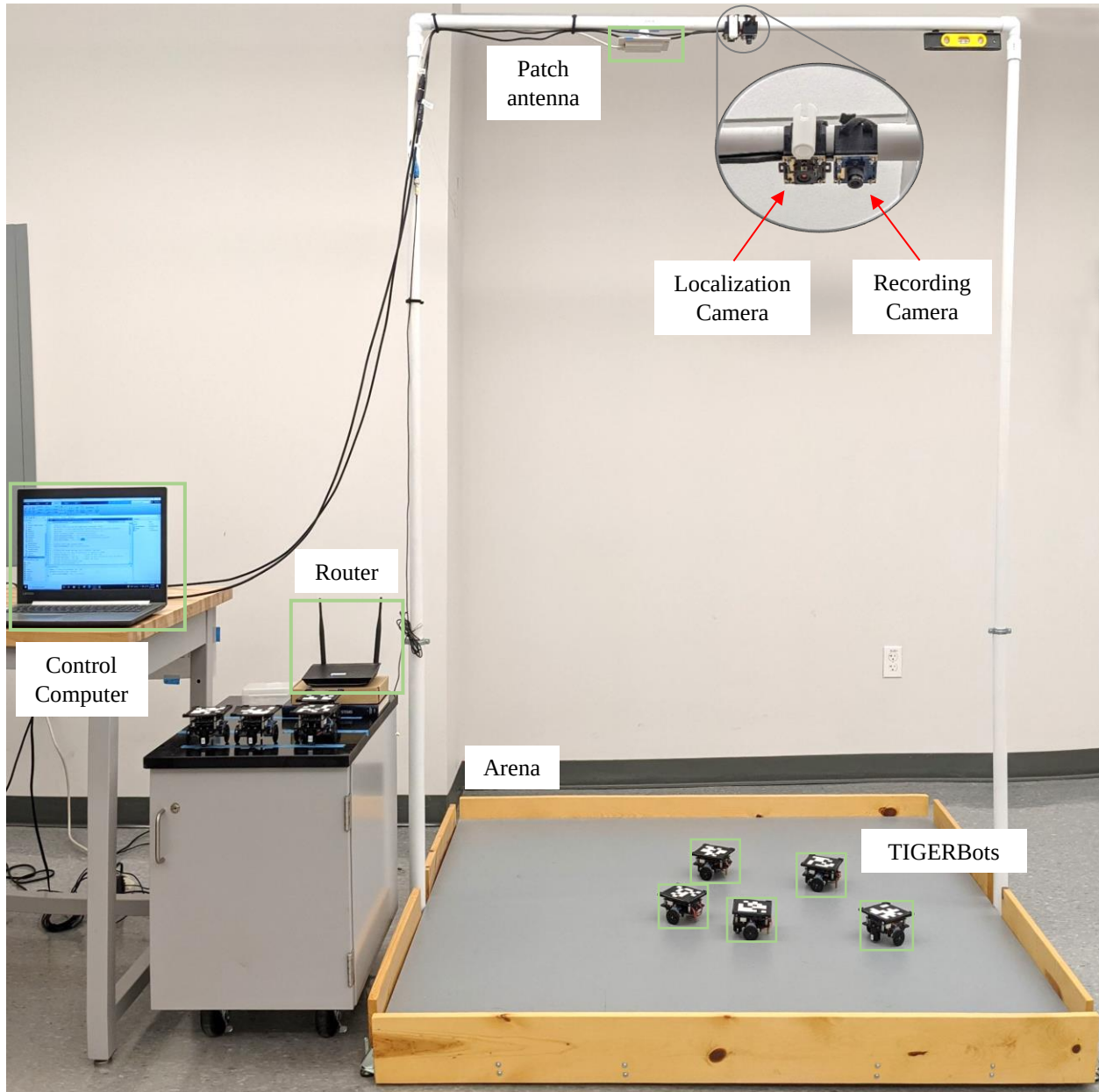


Figure 2.1. TIGER Square test bed and its subsystems. The patch antenna is connected to the router with a co-axial cable, and the router is connected to the computer with a LAN cable.

A MATLAB class file contains all the configurational properties of TIGER Square and high-level macros. During an experimental trial, a top-level script is run which uses this class file to access all other TIGER Square utility scripts. The user is responsible for developing separate scripts for the control algorithms based on a basic skeleton script provided in the controller package.

The control computer can retrieve a top view of the robots in the arena from the overhead localization camera. It utilizes MATLAB Computer Vision Toolbox to detect the AprilTag markers [5] on top of the robots and localize them with respect to a global coordinate frame in real-time; see Figure 2.2.

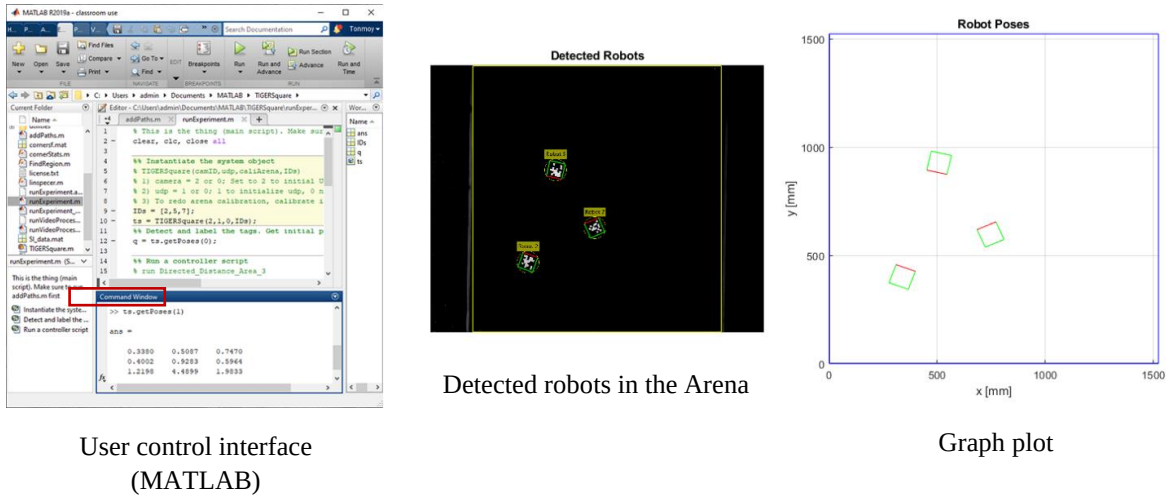


Figure 2.2. Robot localization in the global frame using the control station's image extraction function.

2.3. TIGERBots

TIGERBots are small, wheeled mobile robots that collaboratively carry out control law implementations by forming virtual shapes inside the arena; see Figure 2.3. The size of each robot is approximately 9 x 9 x 9.5 cm. The chassis is made of 3D printed PLA material. The lower base of the chassis houses two stepper motor-wheel combinations, and a Li-Po battery. The top base holds robot's unique ID Tag. The robot's main board and an external IR sensor board on top of it is stacked in between the top base and the bottom base. A steel-ball castor wheel at the front of the robot balances the weight and ensures uniform movements.

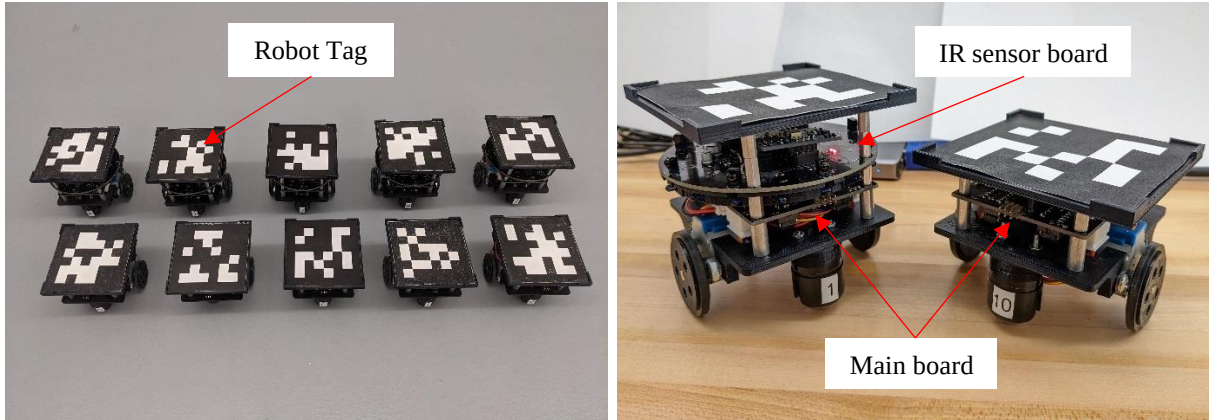


Figure 2.3. TIGERBot fleet; five in the back row equipped with IR sensor board (left image). Individual robots with and without the IR sensor board (right image).

2.3.1. TIGERBot Main Board

TIGERBot's main board coordinates communication, processing, and locomotion from robot's end. The core of the main board is a NodeMCU (ESP 8266 Wi-Fi microchip-built) microcontroller. It controls an ATmega 168P chip that drives the stepper motors. NodeMCU also controls other mainboard circuit components including some introspective sensors. A single-cell 4.2 V 2000 mAh LiPo battery powers each robot. The main board includes an embedded LiPo charging circuit (MCP7383) that operates on a 5 V input.

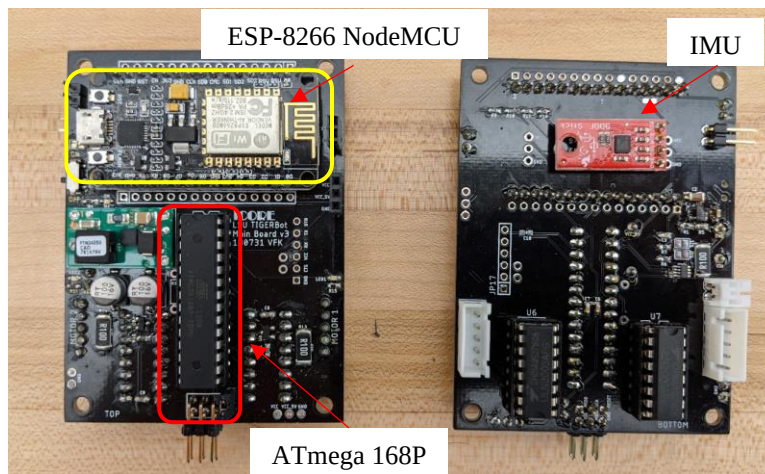


Figure 2.4. TIGERBot main board top view (left) and bottom view (right).

2.3.2. Localization Sensor Board

The localization sensor board is equipped with 8 IR photodiodes ($\pm 60^\circ$ half-angle sensitivity) and 16 LEDs ($\pm 25^\circ$ half-angle intensity) evenly spaced along the circumference of the board [2]. The sensor board circuit is controlled by a Teensy 3.2 microcontroller that coordinates with the mainboard NodeMCU microcontroller for IR signal emitting or receiving and transmitting localization data through Inter-Integrated Circuits (I²C) bus.

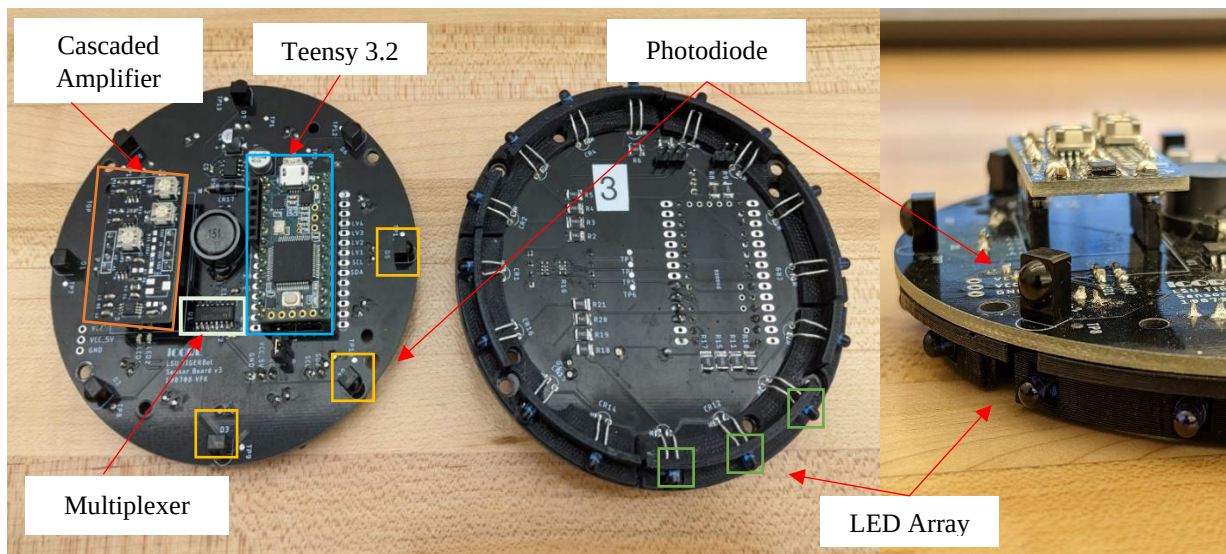


Figure 2.5. TIGERBot IR sensor board components.

When emitting an IR signal, the Teensy activates a LED driver (transistor) to power up the LED array. During IR receiving, a cascaded amplifier circuit consisting of one transimpedance amplifier (TIA) and three sequential non-inverting amplifiers (NIA) converts the photodiode current signals into four distinct voltage signals. This process allows only one photodiode signal to be measured at a time. A turn-based algorithm is used by the Teensy to control a multiplexer to cycle through available photodiodes.

Calibration of Sensor board

The calibration of the sensors is crucial to characterize sensor outputs and achieve reliable localization data. The primary goal for calibration is to set the gain values for the cascading amplifier and normalize the photodiode signals. Two calibration methods are introduced in [19]. The first method is based on individual photodiode signal measurement and rely on a linearly moving LED cart placed on a calibration rail. The second method is implemented by setting one robot as the calibration beacon in the arena, and then averaging the eight photodiode values from each of the surrounding robots as they maintain corresponding calibration distances and rotate in place, all controlled by the MATLAB [19].

2.4. Testbed Performance Synopsis

The performance of various subsystems of TIGER Square was evaluated through a number of stand-alone tests in [19],[22]. The performance parameters were found to be more or less the same during the experiments covered in this work.

Table 2.1. Major performance parameters of the TIGER Square subsystems [19],[22].

Control Station	
• Camera localization accuracy : ± 0.5 cm	
TIGERBots	
• Motor linear speed : Maximum 3.5 cm/s • Battery endurance : Only main board: 1 hr. 20 mins + With sensor board: 25~ 40 mins	
IR Sensor board	
• Effective range : ~ 60 cm • Range accuracy : ± 2.5 cm • Bearing accuracy : $\sim 5.33^\circ$ (Mean absolute error)	

Because of the communication and sensing delay, robot localization process needs to be performed at a predetermined constant refresh rate to maintain predictable trajectory behavior. In other words, a constant step-time for each localization cycle needs to be set. Through trial and

error, step-time for up to 10 robots is recommended in [22] for centralized operations. In [19], step-time for up to 4 robots is recommended for decentralized sensing with centralized communication. The step-time used during the experiments in this work were as recommended in [22] and are listed in Table 2.2.

Table 2.2. TIGER Square Recommended Step-time for Different Number of Robots [22].

No. of robots	1	2	3	4	5	6	7	8	9	10
Step-time (s)	0.14	0.15	0.16	0.18	0.19	0.21	0.22	0.24	0.25	0.27

2.5. Operating Modes

TIGER Square testbed is currently being refined so that it can be operated either in centralized or decentralized formation control mode, including variations. The variations in operating modes are triggered by the absence or unavailability of major control entities; for example, disconnected Wi-Fi router or control computer and/or the localization camera. Specifically, the following four operating modes are considered based on the sensing methods, communication, and processing of the control algorithms.

Mode 1 (Purely Centralized)

In purely centralized operating mode, the control algorithms are processed by the control computer and only the velocity-level control input is passed on to the robots wirelessly in each iteration. Sensing is conducted solely by the overhead localization camera. Each robot communicates only with the control station and execute the control input.

Mode 2 (Centralized Sensing and Communication, Decentralized Processing)

In this operating mode, sensing is still done by the overhead camera with respect to the global coordinate frame. Communication is still centralized, but the control station sends only localization information to the robots instead of velocity commands. Each robot receives the localization information but processes its own control algorithm and execute the velocities.

Mode 3 (Centralized Sensing, Semi-Centralized Communication, and Semi-Centralized Processing)

This operating mode can be considered as a variant of *Modes 1* and 2, specifically aimed at demonstrating some level of inter-robot (semi-centralized) communication. In this mode, the control station sends robot pose information only to one of the robots (called the *communication leader*). The leader then processes the control algorithms and then sends velocity commands to other robots via inter-robot communication protocol. This mode allows us to test the inter-robot communication capability of the TIGERBots without interference from other subsystems.

Mode 4 (Purely Decentralized)

This operating mode is most appropriate when communication with the control station (localization camera and/or Wi-Fi router) is not available or interrupted. Each robot uses its IR sensor board to determine the necessary relative position to other robots and processes its own control algorithm onboard.

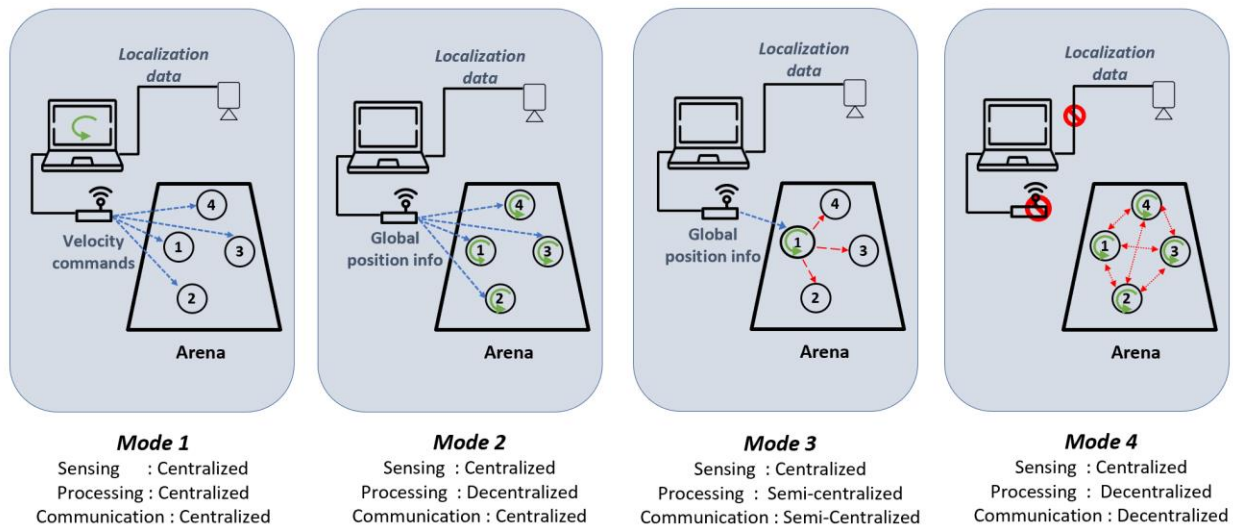


Figure 2.6. TIGER Square Operating Modes layout. The dashed straight arrows indicate the direction of data transfer. The curved arrow indicates whether that device is processing algorithms or not.

Figure 2.6 summarizes the basic layout for all four operating modes. Note that *Mode 2* and *Mode 3* could be considered as intermediate steps towards achieving purely decentralized control (*Mode 4*). We also desire that the testbed has the ability to seamlessly switch between the modes based on the operating conditions without user intervention.

Chapter 3. Decentralized Processing and Communication

With the goal of combining centralized and decentralized sensing, communication, and processing in different operating modes, a unified firmware was introduced to the TIGERBots in this work. Currently with this new version of the firmware, TIGER Square can operate in *Modes 1, 2, or 3* alternately. Once the decentralized sensing techniques [19] are integrated, all four operating modes will be able to work coherently. Therefore, we will limit the discussion in this chapter to the decentralized processing and decentralized communication techniques.

3.1. Communication Protocols

Communication is crucial for wireless distribution of data and coordination of the robots. Henceforth, the communication between the control station and the robots is termed as ‘centralized communication’, while the inter-robot communication method is termed as ‘decentralized communication’. Centralized communication has been widely implemented in various formation control experiments as documented in [19],[22]. Currently, TIGERBots are programmed to initiate all control experiments only after receiving initial user commands from the control station. So, the initialization/interruption of any control experiment in the testbed would depend on centralized communication. As the project progresses towards more autonomy and explores other control laws in complex testing environments, decentralized communication should be fully leveraged.

In the control station, the communication protocol is handled by MATLAB utilizing the Wi-Fi router’s network. On the other hand, in TIGERBots, all the network configurations and data transfer are handled by the main board’s core microcontroller, NodeMCU. This ESP8266-built (Wi-Fi chip) microcontroller is a popular Internet-of-Things device with decent processing power and wide range of wireless applications. It can operate wireless networks in two modes simultaneously - Station (STA) mode and Access Point (termed as software enabled AP or softAP)

mode. In STA mode, NodeMCU can connect to a network as a Wi-Fi station using known network configurations. At the same time, it can establish its own wireless network in softAP mode, which is recognizable by a Service Set Identifier (SSID) [17].

3.1.1 Centralized Communication

In centralized communication, TIGERBots need to connect to the router's network in order to receive any command or send data. Both the control station and the robots can transfer data in form of User Datagram Protocol (UDP) messages. The messages from the control station can be targeted for individual robots. The messages can also be sent to the whole set of connected robots, which is referred to as group messages.

The UDP messages are encoded in JavaScript Object Notation (JSON) format, meaning the contents have a 'key-value pair' structure as used in JSON. The message syntax is converted into text format before they are sent over the wireless network. All the messages from the control station contain two common information: i) message-type ID, and ii) the target robot ID (for group messages, the target ID is set to zero). The message-type ID is predefined based on what information is being sent (e.g., velocity commands, robot sensor data etc.). These IDs are declared inside of both the control software and the robot firmware.

All UDP messages are broadcasted to a UDP port using a broadcasting IP address. Each robot listening to that same UDP port can receive those messages simultaneously. Broadcasting group messages is faster than delivering individually targeted messages. Once received, the robots need to parse (decode) the JSON messages to extract the desired data based on the 'key-value pair' contents, whether they are individual or group messages.

Figure 3.1 summarizes the centralized communication steps.

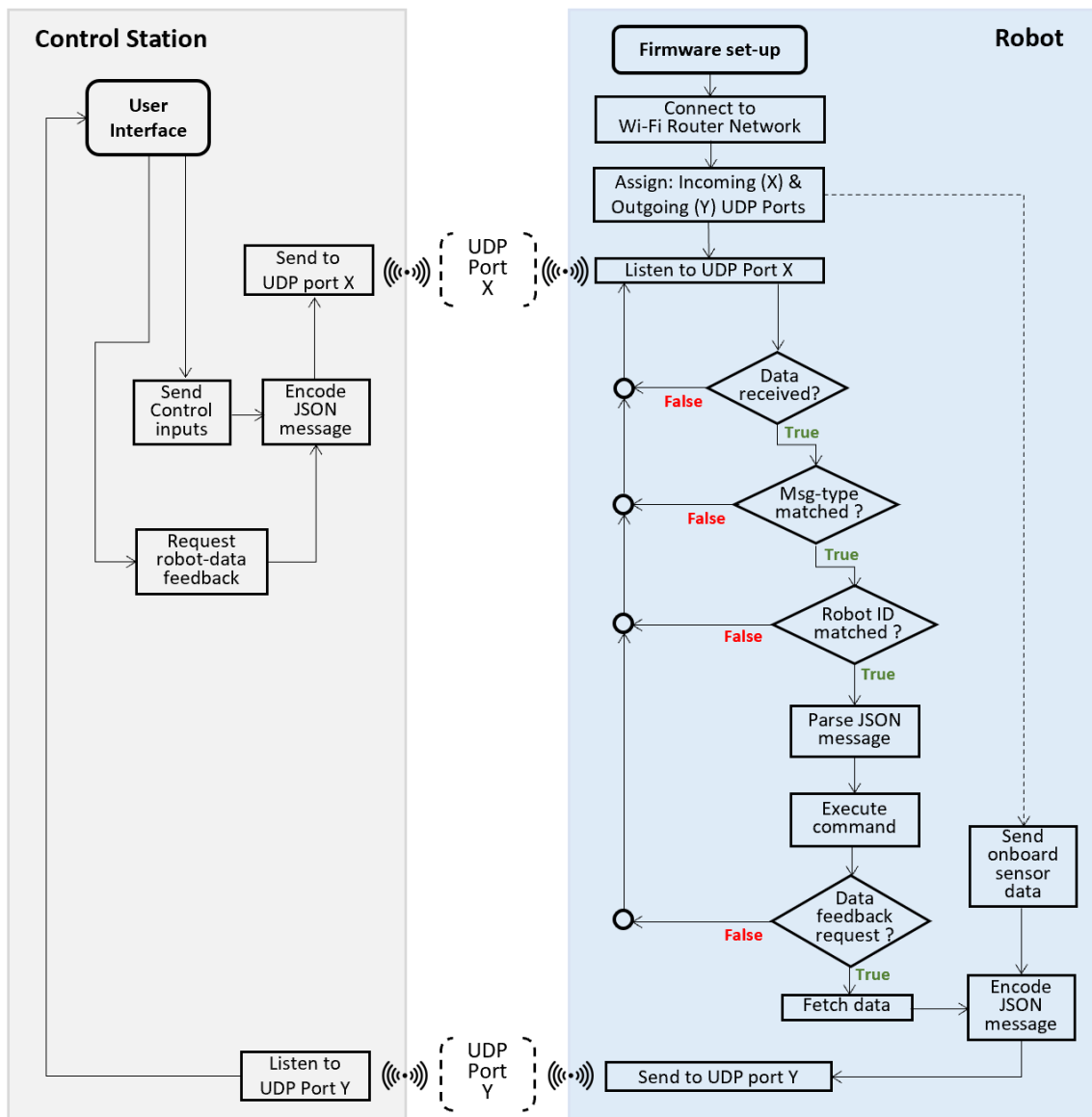


Figure 3.1. Centralized communication working flowchart. The Wi-Fi symbols close to specific blocks indicate that wireless data transfer is involved.

3.1.2. Decentralized Communication

The decentralized communication involves only the robots. This inter-robot communication is established by using ESP-NOW, a communication protocol developed by Espressif for ESP modules [5]. It utilizes a low-power 2.4GHz peer-to-peer wireless connectivity (similar to Wi-Fi channels) without the need of a Wi-Fi router or same WLAN network. According to the developers, the protocol currently supports transmission up to 250 bytes of data and 20 unencrypted peers (ESP modules) depending on pairing configuration.

Prior to data transfer, the NodeMCU boards in the robots are required to be paired with each other using ESP-NOW functions. The role of the individual robot also needs to be set as a ‘controller’ (sender) or a ‘slave’ (receiver) or combined. For consistency, the robot with the smallest robot ID is always assigned as the controller (*communication leader* in *Mode 3*), and the other active robots are assigned as slaves. ESP-NOW allows both one-way and two-way communication between the ESP modules, so the data transfer direction can be altered according to user’s choice or mission profile.

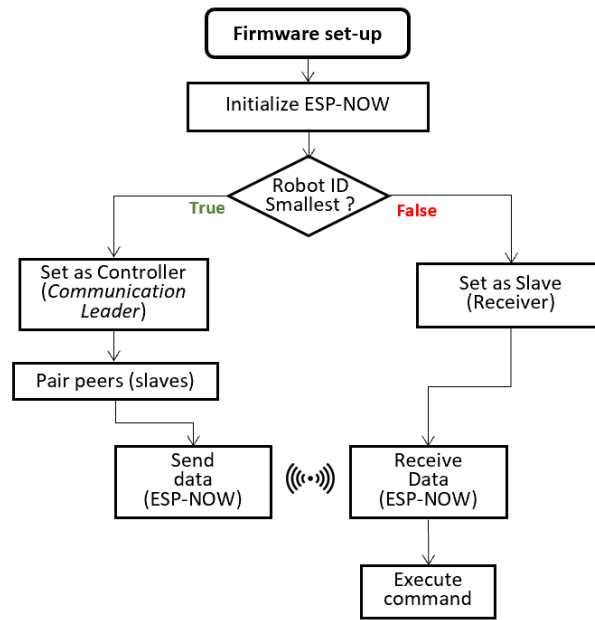


Figure 3.2. Decentralized communication working flowchart.

3.1.3. Wi-Fi Communication-based Active Robot Detection

The control station needs to detect active robots in the arena during every experiment as an initialization step. A UDP heartbeat message from each assigned active robot containing their robot ID and battery voltage confirm their activated or deactivated state. Active-robot detection can alternatively be done by leveraging robots' wireless scanning capabilities.

To implement the scanning-based detection, each robot is configured in soft-AP mode with pre-assigned unique network settings. The firmware then runs the Wi-Fi scanning function to detect active robot networks (SSID) and filter out any other unwanted network. Once all the active robots are detected, the information could be used for decentralized communication or experiment initialization. However, this is not a localization method. Besides, a few limitations refrain this method from fully replacing the camera-based initialization:

- The scanning takes about 3~7 seconds to complete, which could be considered as slow.
- The scanning would detect any active robot nearby, even if they are not inside the arena or part of the experiment.

Nevertheless, this method poses some interesting features for future decentralized applications.

Both centralized and decentralized communication protocols include some delay during the execution. Preliminary stand-alone tests verify that the most time-consuming segment of each centralized communication cycle is constructing the UDP objects in MATLAB, and not the router's communication with the robots. These delays are directly responsible for the step-time values shown in section 2.4. Decentralized communication, on the other hand, evidently introduces significantly less delay. However, the magnitude of the delay was not established in the present project and is left for future work.

3.2. Decentralized Processing Firmware

TIGERBot main board firmware for centralized operations were introduced in [22]. This firmware was adopted from a similar open-source experimental multi-agent testbed called ‘Robotarium’ [3]. During centralized operations (*Mode 1*), the robots simply need to maintain the network connectivity and listen for UDP messages containing control velocity inputs (described in section 3.1.1). Now, for decentralized processing in *Modes 2* and *3*, all the necessary user inputs need to be set and stored throughout the processing run-time. Once the onboard processors execute the algorithms, the program resets to its centralized state.

3.2.1 Decentralized Processing Initialization

A number of initial user input values need to be sent to the robots to trigger any decentralized control algorithm processing. A single MATLAB script is set in the control computer to send those commands. This script also utilizes the camera feedback for centralized sensing and distance error calculations.

The initial user inputs include but are not limited to a unique algorithm ID, the desired run-time, and control gains. Other user inputs are algorithm-specific, such as initial target positions, the formation graph type (directed or undirected), the graph adjacency matrix, and collision avoidance constraints. The algorithm-specific inputs are optional as they have default values set in the firmware. Any user input would just override the defaults, allowing users to experiment with algorithm-specific constraints without reinstalling the robot firmware every time.

The initial input values are stored in the memory as long as decentralized processing takes place. The network configurations and role of the robot (sender or receiver) in decentralized communication are also set during the initialization stage.

3.2.2. Control Algorithm Processing

The controller codes in the firmware for *Modes 2* and *3* have the same skeleton structure as the MATLAB controller algorithm scripts used for *Mode 1*, although there are some key differences in the execution method. Based on the pre-defined step-time interval (Table 2.2), the MATLAB controllers determine the total number of iterations to run for a given run-time period. During each iteration, the MATLAB controllers use the robots' global position information to calculate velocity control inputs, and then send those to the robots.

The robot's onboard controller codes on the other hand are simply executed within the experiment run-time period. The control algorithms are coded as user-defined functions in the firmware, which only start working if the UDP command messages initiate them. The nested loops inside those functions break out when the execution period exceeds the desired run-time. At the end of the completion of any algorithm, all the initial user inputs are reset to their default values. Figure 3.2 presents a flowchart of the working method for the robot firmware in *Modes 1,2* and *3*.

This run-time bounded execution method in decentralized processing is chosen for two reasons. First, it prevents the program from getting stuck inside an infinite loop. Second, the firmware does not need to know the step-time interval. This method also led to an interesting finding. Given the same conditions, the NodeMCU completes each iteration faster than the MATLAB controller. The reason, which requires further investigation, could be because MATLAB needs to pull so many resources while executing the code. However, the shorter processing time of the NodeMCU can help minimize the convergence time of the robots during experiments.

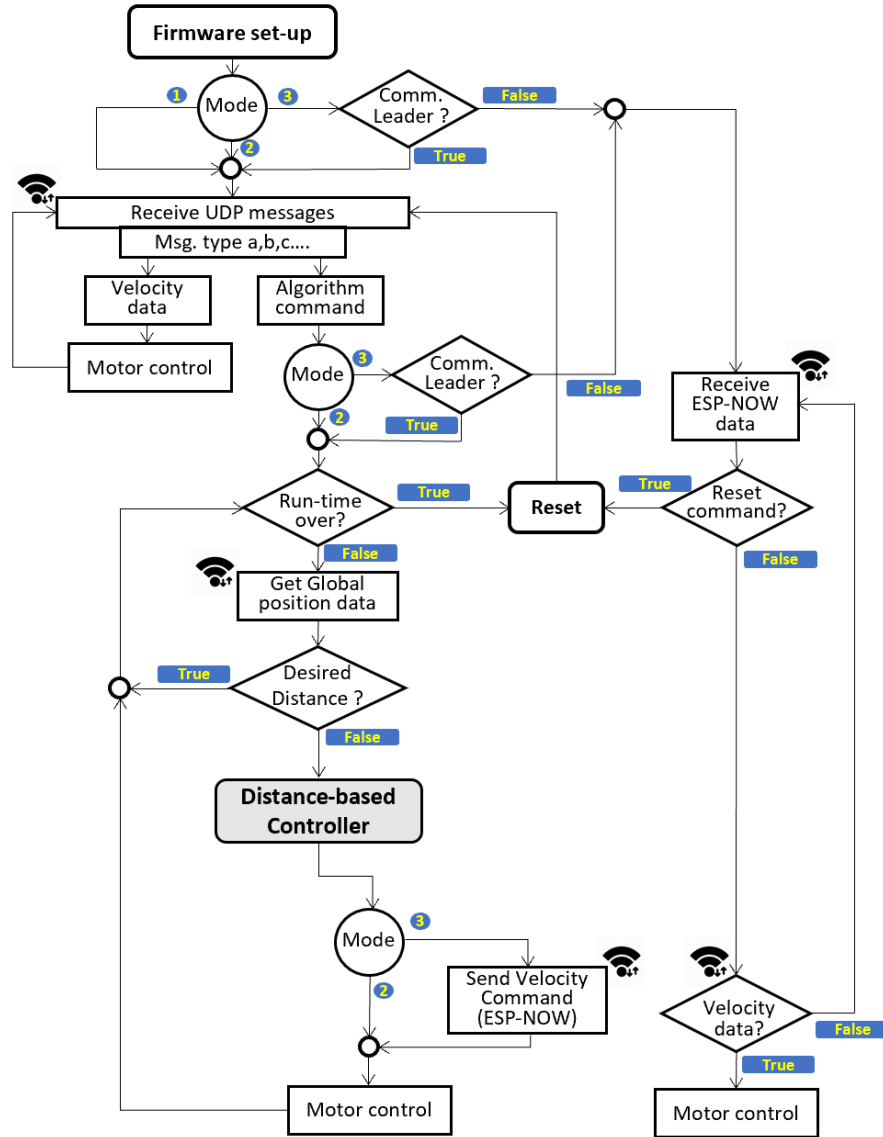


Figure 3.3. TIGERBot Firmware simplified workflow in *Modes 1, 2 and 3* combined. Only higher-level tasks are shown, while background tasks are skipped. The Wi-Fi symbols close to specific blocks indicate that wireless data transfer is involved.

Firmware Utility Library

In order to produce executable linear and angular velocity outputs for the robots, the control algorithms need to map single-integrator equations into nonholonomic, unicycle model velocities or vice versa (details in section 4.1). A user-defined library was introduced to the decentralized

firmware that is dedicated to all the kinematic model transformations along with some other essential utility functions described in the next paragraph.

Robots are frequently required to move to specific global coordinates inside the arena. The library includes a control algorithm named ‘Parking Controller’ that drives the robots to the desired position and orientation within error bounds. It includes another function for generating virtual polygon shapes with respect to arena coordinates, depending on the number of robots. Both algorithms were implemented previously in the MATLAB centralized controller [22]. Besides these algorithms, the library also includes a handful of macros for various mathematical operations required for the decentralized processing of the control algorithms.

This user-defined library is written in C++, which includes a few NodeMCU, Arduino Core libraries, and is currently only functional with the TIGER Square firmware. A popular C++ math library ‘Eigen’ was used to perform most matrix operations required by the control algorithms. A few reasons for choosing ‘Eigen’ are that it is easy to install, it performs complex mathematical operations very fast, and it has MATLAB-like syntaxes.

Over The Air (OTA) Program Upload

In order to reflect any change in the firmware, the program needs to be uploaded to the robots’ microcontrollers every time. This requires physically connecting each robot to the computer with a USB cable. The process is time consuming, especially when there are multiple robots involved. A viable solution to this is the use of an OTA (Over the Air) update function for the ESP modules, which enables users to upload firmware using the wireless network rather than a serial port (USB). Such functionality becomes extremely useful in case of limited or no physical access to the module.

Chapter 4. Formation Control Experiments

This chapter first outlines the theory behind the formation control algorithms that this work will experimentally implement. In later sections, we present a set of experimental results for these implementations. The implementations are not necessarily new since the algorithms were previously tested in [22] in purely centralized manner (*Mode 1*). Rather, the goal is to test the performance of the modified TIGERBot firmware with onboard control algorithm processing. Specifically, we seek to demonstrate that no deterioration occurs in the formation performance when the system operates with decentralized processing.

4.1. Graph Theory and Formation Control Problems

Multi-agent systems can be described conveniently using rigid graph theory. A graph, G , is a pair (V, E) where, $V = \{1, 2, \dots, n\}$ is the set of nodes and $E \subset V \times V$ is the set of edges. In the context of distance-based multi-agent formation control [3], the nodes represent the agents, and the edges represent the distance between the agents. A graph with n nodes and l edges where $l = 2n - 3$ is said to be minimally rigid [3]. For any agent i , the set of its neighboring agents is denoted by

$$N_i(E) = \{j \in V \mid (i, j) \in E\} \quad (4.1)$$

To represent tracking direction between any two agents along an edge, we consider two categories of graphs, termed ‘directed’ graph and ‘undirected’ graph. In the directed case, an edge length or inter-agent distance is controlled by only one agent, while in the undirected case both agents along an edge attempt maintain the distance. The direction of tracking in a directed graph is presented by edge with single arrowhead. Arrowheads in both ends of an edge means bidirectional tracking or alternately undirected relation between the agents. Note that undirected

or bidirected graphs are often represented by edges without any arrowheads. Figure 4.1 illustrates directed and undirected tracking connections between two and three agents.

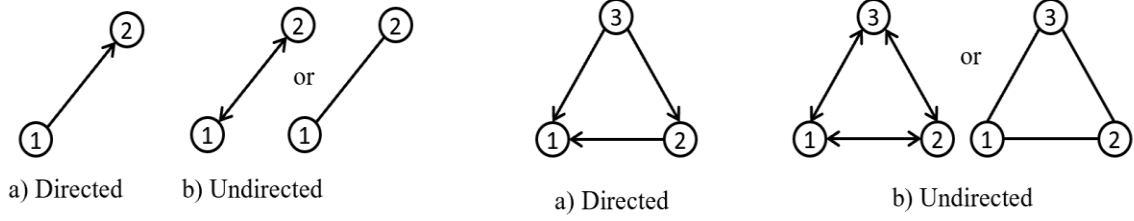


Figure 4.1. Directed and undirected graphs between 2 agents and 3 agents, respectively.

Single-Integrator model

The kinematic model that would be used for the agents is called the ‘single-integrator’ model, which is derived from the omnidirectional robot model [8],[16]. This model consists of only two variables: position and velocity. For a system with n number of agents, the single-integrator model can be expressed as the following first-order differential equation

$$\dot{\mathbf{q}}_i = \mathbf{u}_i \quad (4.2)$$

where $\mathbf{q}_i$ is the position and $\mathbf{u}_i$ is the velocity-level control input of the i th agent with respect to an Earth-fixed coordinate frame.

4.1.1 Formation Acquisition

The formation acquisition problem requires the agents to acquire and maintain a virtual predefined geometric shape in space. To mathematically represent the problem, we first define the position vectors of i th and j th agents as $\mathbf{q}_i$ and $\mathbf{q}_j$ respectively, in a global coordinate frame. Their relative position, $\tilde{\mathbf{q}}_{ij}$ is given by

$$\tilde{\mathbf{q}}_{ij} = \mathbf{q}_i - \mathbf{q}_j \quad (4.3)$$

If the desired distance between the two agents is given by d_{ij} , the distance error, e_{ij} , is given by [3]

$$e_{ij} = \|\tilde{\mathbf{q}}_{ij}\|^2 - d_{ij}^2 \quad (4.4)$$

The control law for the i th robot to maintain a distance of d_{ij} from the j th robot is given by [3]

$$\mathbf{u}_{i_{acq}} = -k_v \sum_{j \in \mathbb{N}} \tilde{\mathbf{q}}_{ij} e_{ij}, \quad i = 1, 2, \dots, n \quad (4.5)$$

where $k_v > 0$ is a user-defined control gain. This control law has been proven to ensure the local exponential stability of the equilibrium point $e_{ij} = 0$ [16].

4.1.2. Formation Maneuvering

Formation maneuvering problem requires the agents to simultaneously acquire a formation and maneuver cohesively as a virtual rigid body along a predefined trajectory. Mathematically, the objective for this problem can be expressed as

$$\dot{\mathbf{q}}_i(t) - \mathbf{v}_{di}(t) \rightarrow 0 \text{ as } t \rightarrow \infty, \quad i = 1, 2, \dots, n \quad (4.6)$$

where $\mathbf{v}_{di}$ is the desired rigid body velocity composed of a desired translational velocity, $\mathbf{v}_0$, and a desired angular velocity, $\boldsymbol{\omega}_0$. The desired formation velocity is then defined by

$$\mathbf{v}_{di} = \mathbf{v}_0 + \boldsymbol{\omega}_0 \times \tilde{\mathbf{q}}_{in}, \quad i = 1, 2, \dots, n \quad (4.7)$$

The control law that solves the formation maneuvering problem is given by [16]

$$\mathbf{u}_{manu} = \mathbf{u}_{acq} + \mathbf{v}_d \quad (4.8)$$

In (4.8), $\mathbf{v}_d = [\mathbf{v}_{d1}, \dots, \mathbf{v}_{dn}]$ and $\mathbf{u}_{acq} = [\mathbf{u}_{1_{acq}}, \dots, \mathbf{u}_{n_{acq}}]$ where each term $\mathbf{u}_{i_{acq}}$ comes from the formation acquisition problem in (4.5).

This law can be re-written element-wise as

$$\mathbf{u}_{i_{manu}} = -k_v \sum_{j \in \mathcal{N}} \tilde{\mathbf{q}}_{ij} e_{ij} + \mathbf{v}_0 + \boldsymbol{\omega}_0 \times \tilde{\mathbf{q}}_{in}, \quad i = 1, 2, \dots, n. \quad (4.9)$$

Note that, when the virtual rigid body velocity $\mathbf{v}_{di}$ in (4.7) only includes translational component, the formation maneuvering problem is referred as *flocking*. All the maneuvering experiment results presented in section 4.2.2 is implementation of *flocking* problem. On top of that, the flocking velocity is considered constant throughout the run-time.

4.1.3. Nonholonomic Unicycle Model to Single-Integrator Transformation

We consider that $\{X_0, Y_0\}$ is a fixed, global reference frame and local reference frame, $\{X_i, Y_i\}$, is attached to the i th robotic vehicle with a heading direction, θ_i , relative to X_0 (Figure 4.2). Here, C_i is the vehicle's geometric center assumed to coincide with its center of rotation.

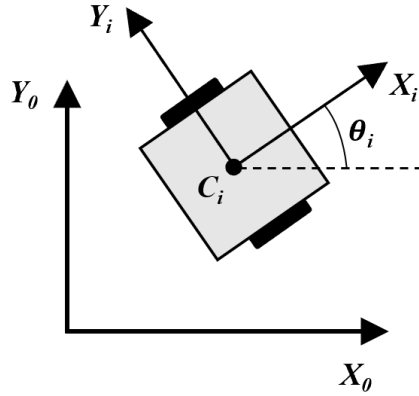


Figure 4.2. Robotic vehicle model with unicycle dynamics.

The robot's movement can be expressed by the following unicycle, nonholonomic kinematic model [16].

$$\dot{q}_i = \begin{bmatrix} \cos \theta_i & 0 \\ \sin \theta_i & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_i \\ \omega_i \end{bmatrix} \quad (4.10)$$

where v_i is robot's translational speed along θ_i direction and ω_i is the angular speed about the vertical axis passing through robot's center of mass. As (4.10) is a single-integrator-like equation, we can apply the following transformation to get the value of desired control velocity input, u_i [16].

$$\dot{q}_i = \begin{bmatrix} v_i \cos \theta_i \\ v_i \sin \theta_i \end{bmatrix} =: u_i = \begin{bmatrix} u_{ix} \\ u_{iy} \end{bmatrix} \quad (4.11)$$

And
$$\dot{\theta}_i = \omega_i \quad (4.12)$$

The heading error, $\tilde{\theta}_i$ can be defined as

$$\tilde{\theta}_i = \theta_i - \theta_{di} \quad (4.13)$$

where θ_{di} is the desired heading angle represented by

$$\theta_{di} = \begin{cases} 0, & \text{if } u_{ix} = u_{iy} = 0 \\ \tan^{-1} \left(\frac{u_{iy}}{u_{ix}} \right), & \text{otherwise} \end{cases} \quad (4.14)$$

and,
$$\dot{\theta}_{di} = \begin{cases} 0, & \text{if } u_{ix} = u_{iy} = 0 \\ \frac{u_{ix}}{u_{ix}^2 + u_{iy}^2} \dot{u}_{iy} - \frac{u_{iy}}{u_{ix}^2 + u_{iy}^2} \dot{u}_{ix}, & \text{otherwise} \end{cases} \quad (4.15)$$

Finally, the control law for the nonholonomic kinematic model can be expressed as [16]

$$v_i = u_{ix} \cos \theta_i + u_{iy} \sin \theta_i \quad (4.16)$$

$$\omega_i = -\beta_i \tilde{\theta}_i + \dot{\theta}_{di}, i \in V_d \quad (4.17)$$

where $\beta_i > 0$ is a constant control gain.

4.1.4. Framework Ambiguity

As only inter-agent distances are regulated in the distance-based formation control scheme, the formation can converge to any isometry of the geometric shape. This phenomenon is referred to as framework ambiguity. Two common types of ambiguities called ‘Reflected ambiguity’ and ‘Flip ambiguity’ can be seen in Figure 4.3.

One way of avoiding the ambiguity is to initialize the agents sufficiently close to their desired formation, which leads to local stability only. Another solution is adding a signed area constraint to the control law and using a directed graph [20]. This second solution is not tested in this work.

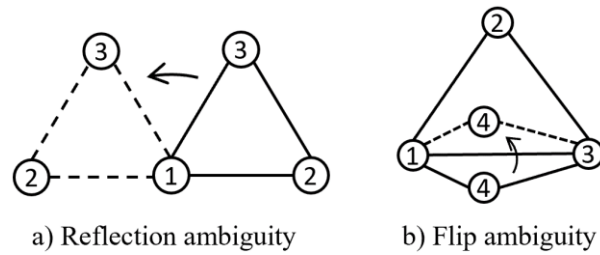


Figure 4.3. Framework ambiguity examples.

4.2. Experimental Results

The formation acquisition and formation maneuvering control laws described in the previous section were implemented in both *Mode 1* and *Mode 2*. The convergence time for the desired formation, and the quality of the inter-agent distance errors were the main performance indices for comparing the operating modes. Note that satisfactory results from *Mode 2* are enough to evaluate the performance of decentralized processing compared to the centralized scheme. As *Mode 3* solely aims at establishing decentralized communication, only one set of experiments is presented to justify whether the robots could achieve the desired formation. For each case, several trials were run to rule out any undesired or inconsistent behavior during the experiments.

The movements of the robots during each trial are presented by post-processed undistorted image frames captured by the secondary recording camera. The frames are annotated with robot number labels, inter-robot tracking connections, and robot trajectory trails. The MATLAB controller stores the localization data in each iteration, which is used to produce the error graphs and the trajectory plots of the robots. Note that the error data in *Modes 2* and *3* are based on camera data, and not onboard sensor data. Also notice that small spikes occasionally appear in the distance error plots in Figures 4.4 to 4.10. This indicates the robot missed steps most likely due to not receiving wireless commands. However, they usually recovered in the next steps.

4.2.1 Formation Acquisition Experiments

Formation acquisition experiments were done with 3, 4 and 6 robots. In each experiment, only undirected graphs were used; the robots' initial conditions and the control gains were kept the same for proper comparison. Only the experiment with 4 robots was done in *Mode 3* to verify the feasibility of decentralized communication.

Table 4.1. List of Formation Acquisition Experiments.

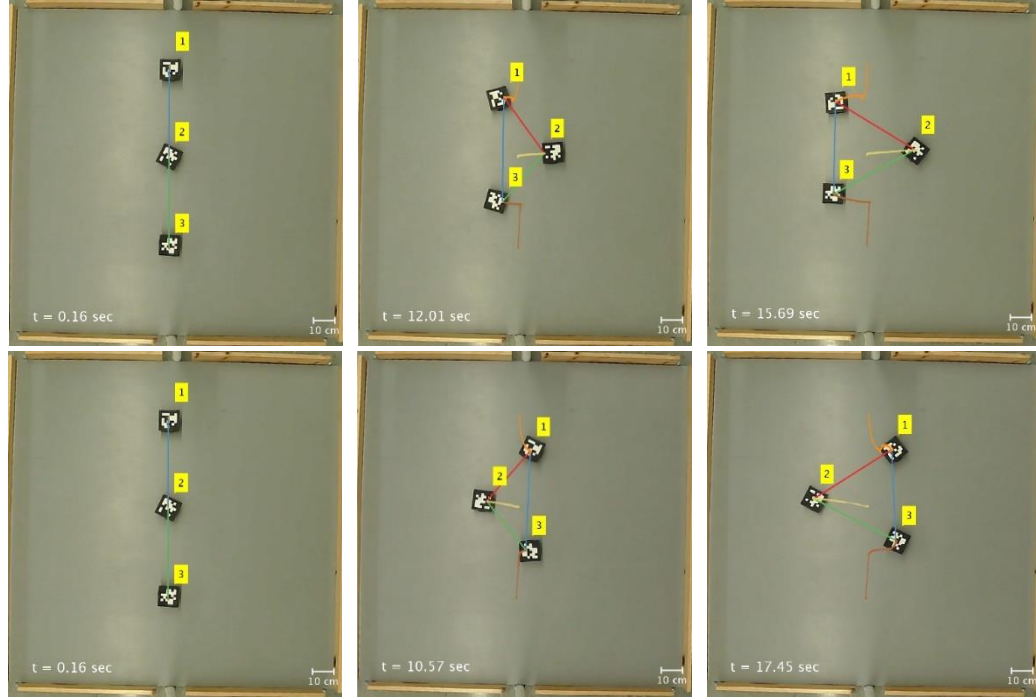
	No. of robots	Initial conditions	Desired Shape	Operating Modes
Set 1	3	Co-linear	Triangle	1,2
Set 2	4	Random	Rigid Square	1,2,3
Set 3	6	Random Hexagon	Triangle	1,2

The results for each formation acquisition experiment can be found in Figures 4.4 to 4.6. In all cases, the robots successfully acquired the desired formation shape. The distance error graphs indicate that the errors eventually reach approximately zero. The small oscillatory, steady-state behavior about zero is expected due to the camera sensing resolution and noise. Therefore, we considered that convergence occurred whenever all the errors reached the oscillatory region. The values of approximate convergence time and the average steady-state error for all the formation acquisition experiments can be found in Table 4.2.

The most interesting results are obtained in Figure 4.4 with 3 robots. As the robots are initially positioned co-linearly, they should not be able to leave their positions using the distance-based controller. However, due to sensing resolution and measurement error, they move and finally converge to their desired shape. When compared, the final shapes in *Mode 1* and *Mode 2* clearly exhibit the reflection framework ambiguity.

Mode 1

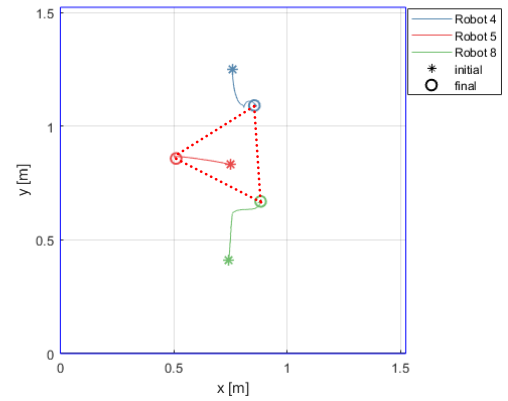
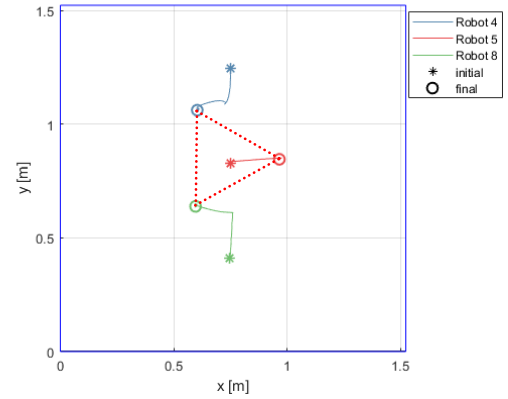
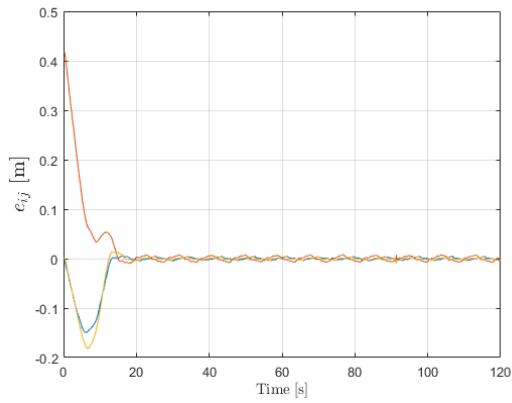
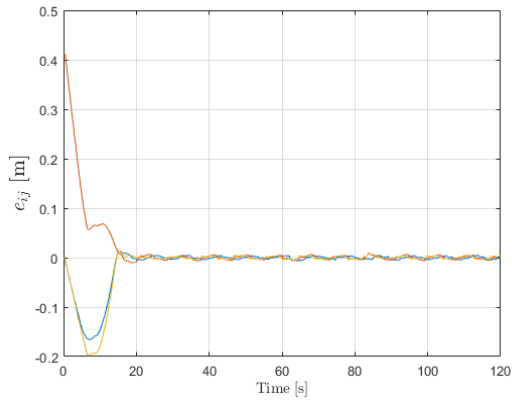
Mode 2



i) Movement from initial co-linear positions to desired triangular formation (left to right)

Mode 1

Mode 2

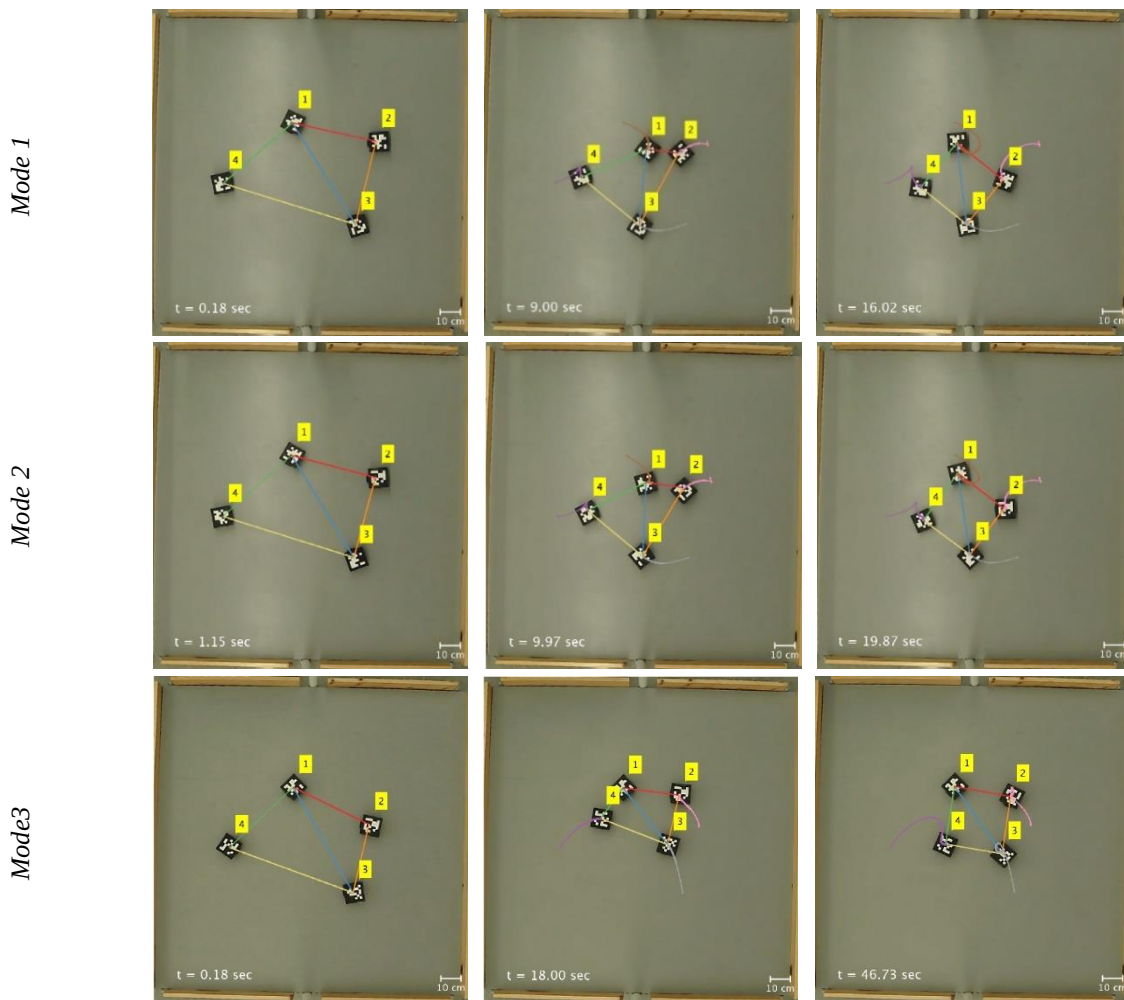


ii) Distance errors

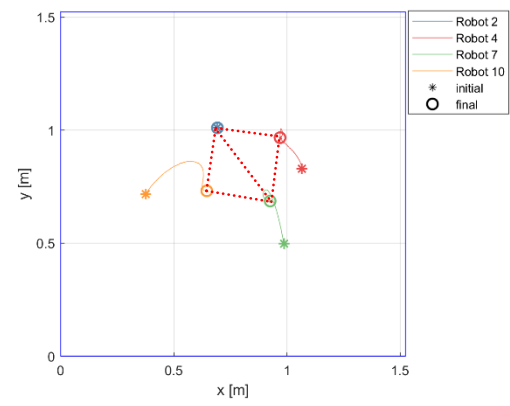
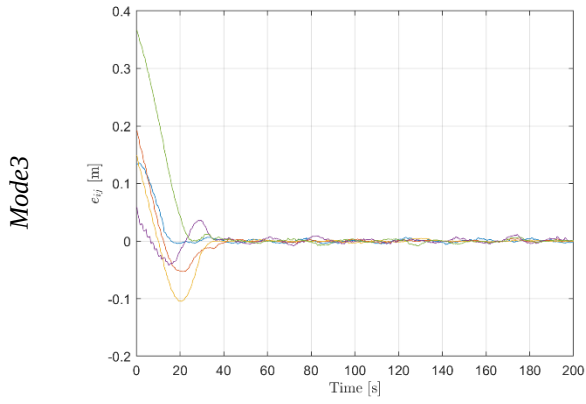
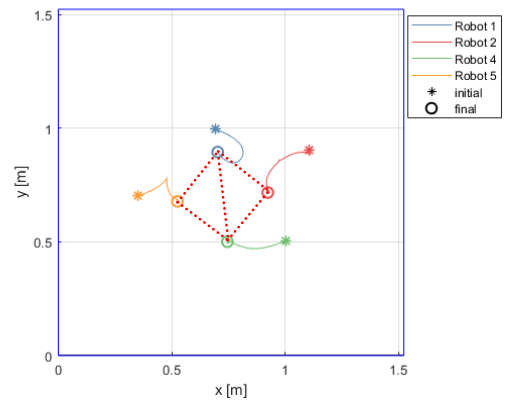
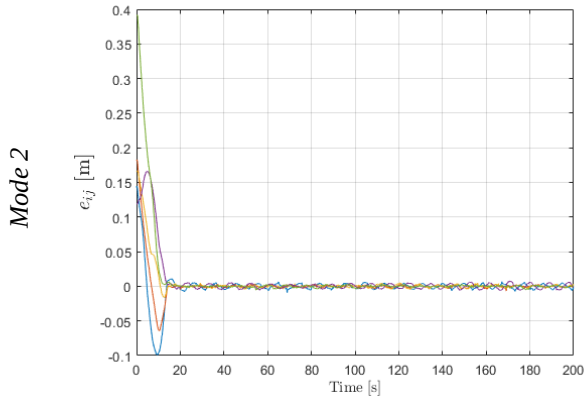
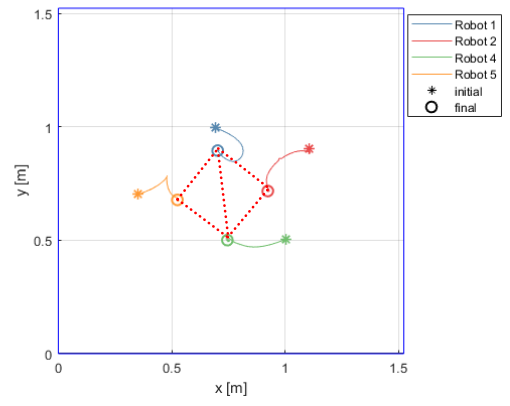
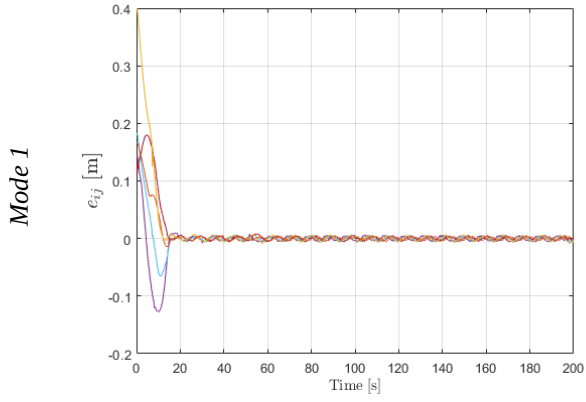
iii) Robot paths

Figure 4.4. Formation acquisition with three robots.

In the formation acquisition experiment with 4 robots in *Mode 3*, only the communication leader (robot 1 in Figure 4.5) receives the position feedback from the control station. The other three robots are not connected to the centralized network. The leader completes the processing and sends the velocity commands to all the follower robots via the decentralized communication protocol (ESP-NOW). The followers just receive the velocity commands from the leader and execute them. It is clear from Figures 4.5 that the robots converge to their desired square formation in all three modes. The approximate convergence time and the average steady-state error are included in Table 4.2.



i) Movement from initial positions to desired square formation (left to right).

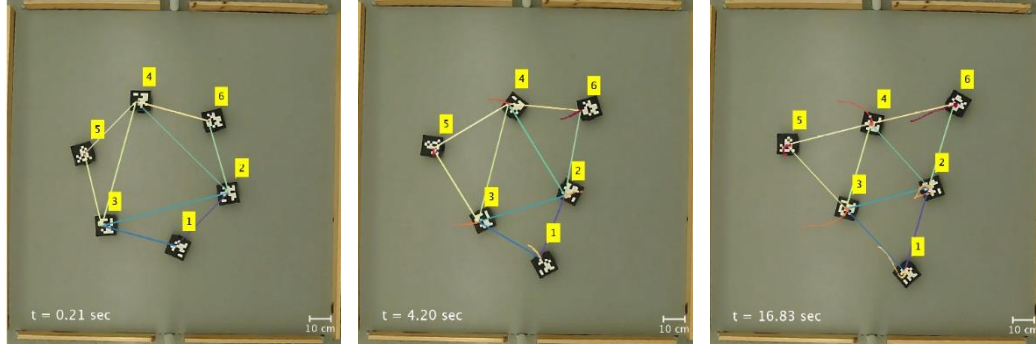


ii) Distance errors

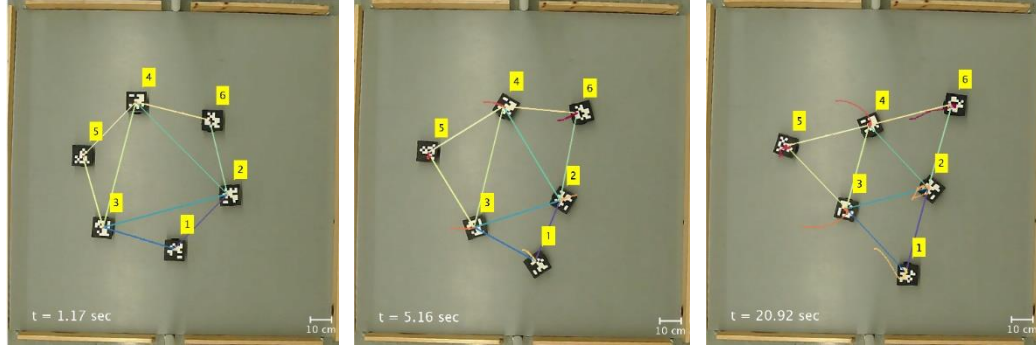
iii) Robot paths

Figure 4.5. Formation acquisition with four robots.

Mode 1

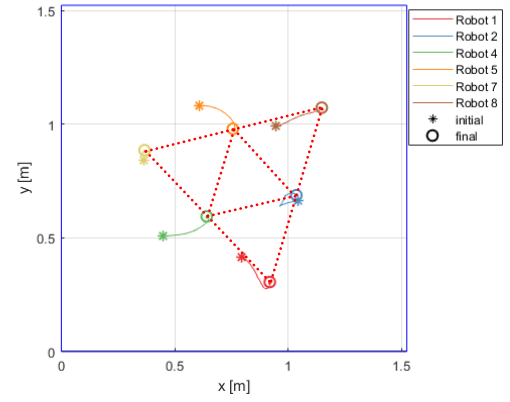
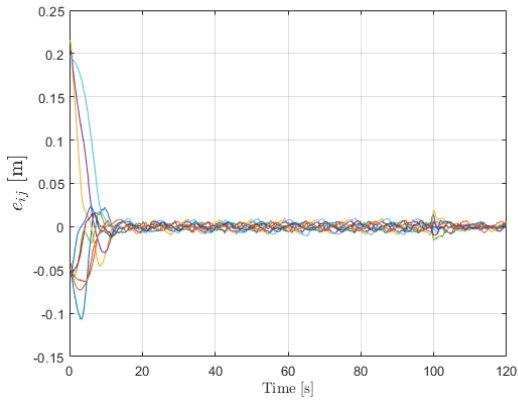


Mode 2

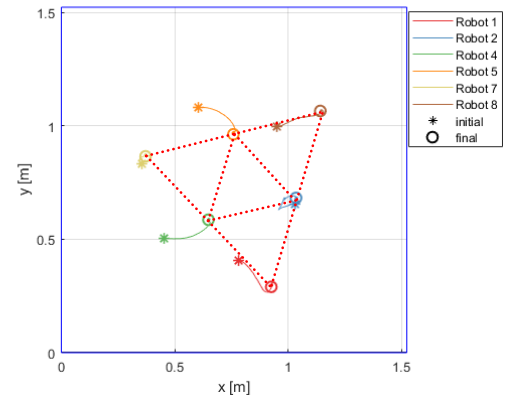
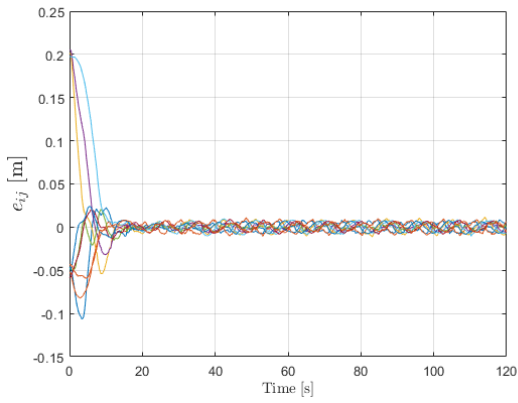


i) Movement from initial sexagon positions to desired triangular formation (left to right)

Mode 1



Mode 2



ii) Distance errors

iii) Robot paths

Figure 4.6. Formation acquisition with six robots.

Table 4.2. Summary of Formation Acquisition Experiments.

No. of robots	3		4			6	
Operating Modes	<i>Mode 1</i>	<i>Mode 2</i>	<i>Mode 1</i>	<i>Mode 2</i>	<i>Mode 3</i>	<i>Mode 1</i>	<i>Mode 2</i>
Convergence Time(s)	15.04	19.36	19.26	19.62	41.22	17.43	19.11
Steady-State Error range (cm)	[-1.04, 1.45]	[-0.97, 0.81]	[-0.79, 0.8]	[-0.91, 0.76]	[-0.84, 1.17]	[-1.56, 1.92]	[-1.13, 1.17]
Average Steady-State Error (cm)	0.29	0.26	0.27	0.18	0.19	0.32	0.39

In general, the average convergence time for the robots were found to be higher in *Mode 2* than in *Mode 1*, and highest in *Mode 3*. However, the average steady-state error with 3 and 4 robots were found to be less in *Mode 2* compared to *Mode 1*.

The larger convergence time in *Mode 3* is however expected, and likely because of the combined communication delays of the centralized and decentralized protocols. If the follower robots are off course for an instance, the recovery is prolonged due to the combined communication delays, as seen in the error graph (Figure 4.5.ii).

4.2.2. Formation Maneuvering Experiments

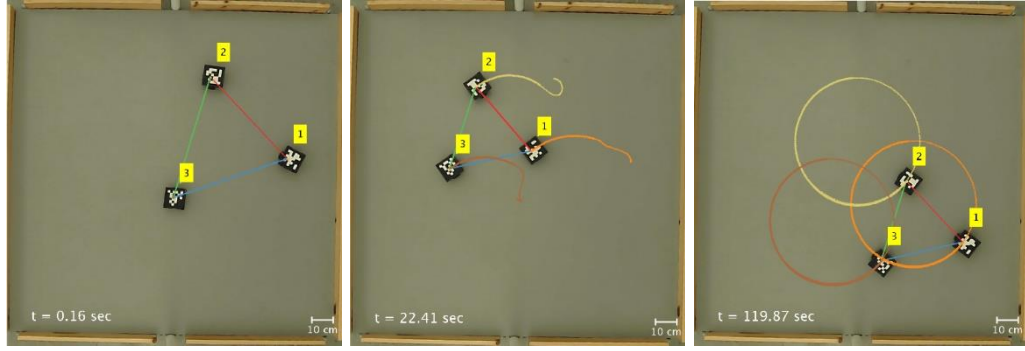
The formation maneuvering experiments were done with up to 9 robots and with undirected graphs only. In all the experiments, robots were required to acquire the desired polygonal formation and then maneuver as a virtual rigid body around a virtual circle. Again, the same initial positions and control gains were used in both *Mode 1* and *Mode 2* for proper comparison.

Table 4.3. List of Formation Maneuvering Experiments.

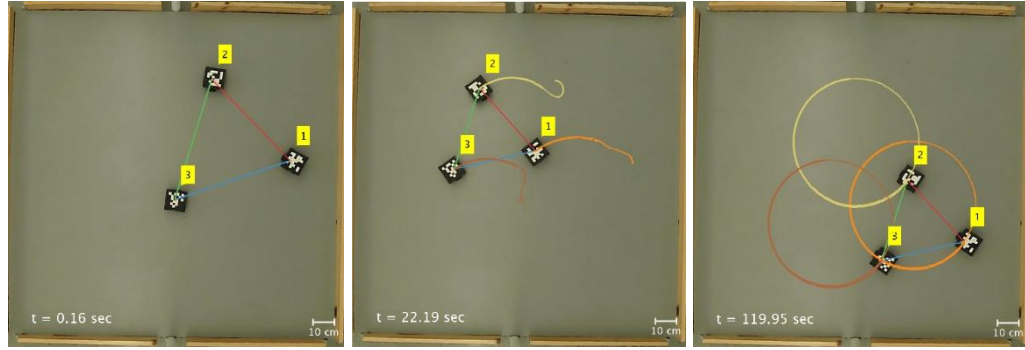
	No. of robots	Initial conditions	Desired Shape
Set 1	3	Triangle	Triangle
Set 2	4	Square	Square
Set 3	6	Hexagon	Hexagon
Set 4	9	Nonagon	Nonagon

In all cases, the error graphs exhibited oscillatory behavior once they reached the steady-state near zero error (see Figure 4.7.ii through Figure 4.10.ii). The same applies for the heading angle errors. In both operating modes, the performance of the formation degraded with the increase of number of robots. The convergence times and average steady-state error values for the maneuvering experiments can be found in Table 4.4.

Mode 1

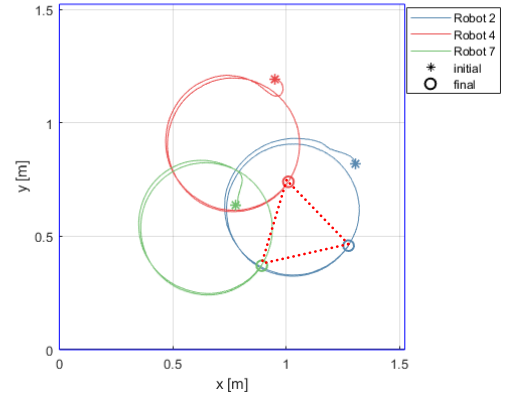
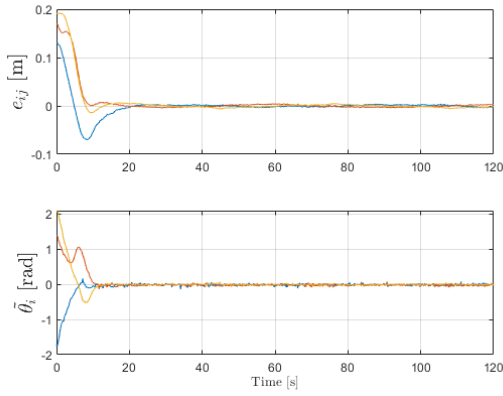


Mode 2

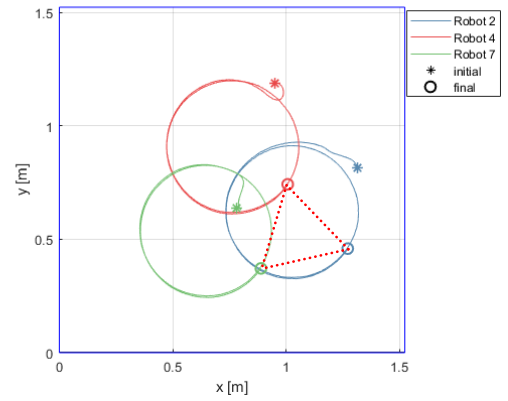
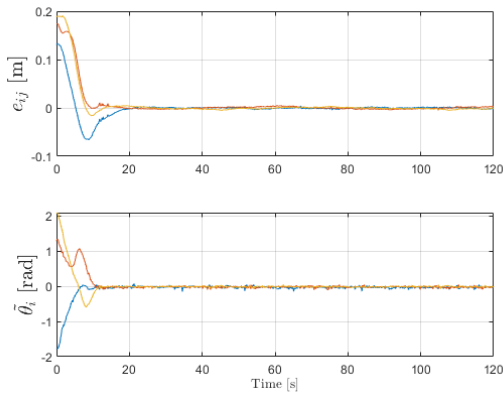


i) Translation from initial conditions to desired formation (left to right)

Mode 1



Mode 2

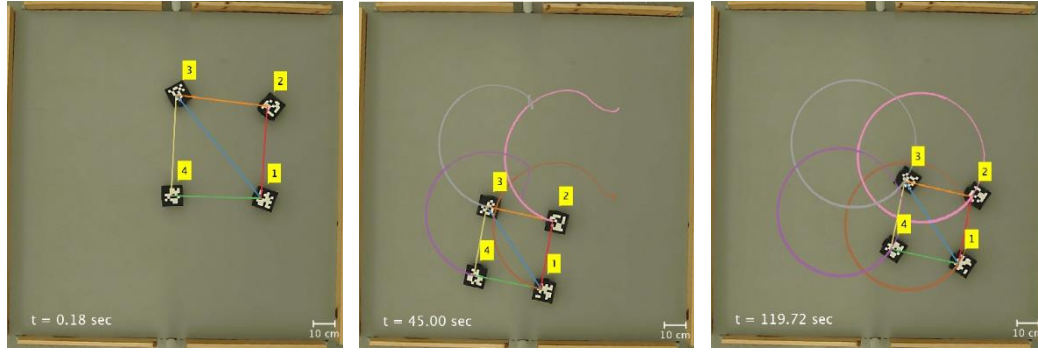


ii) Distance and heading angle errors

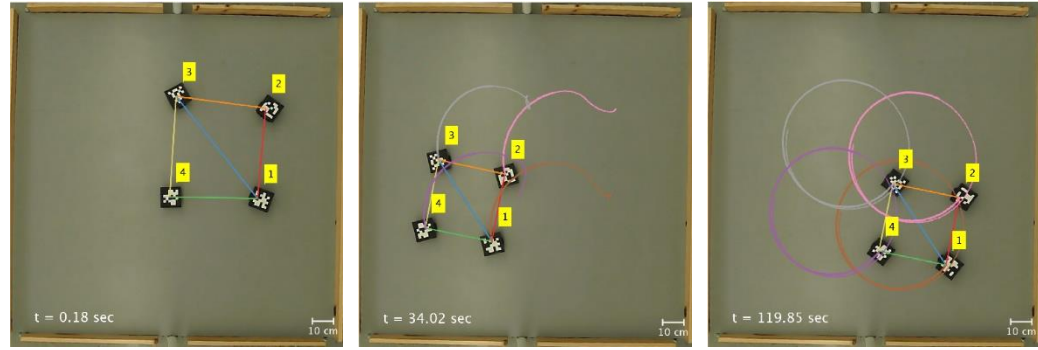
iii) Robot paths

Figure 4.7. Formation maneuvering with three robots.

Mode 1

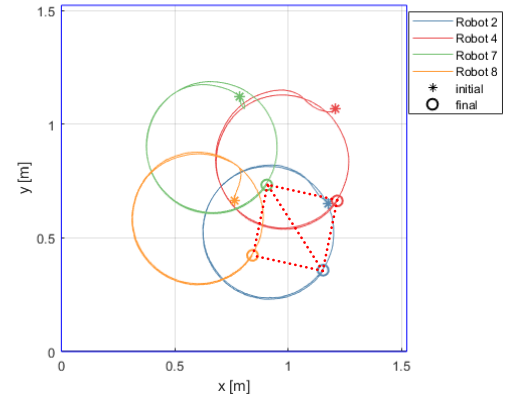
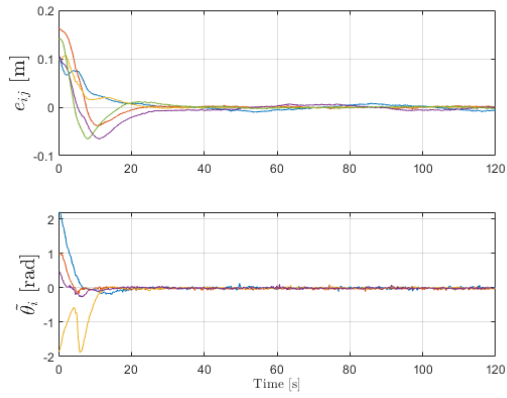


Mode 2

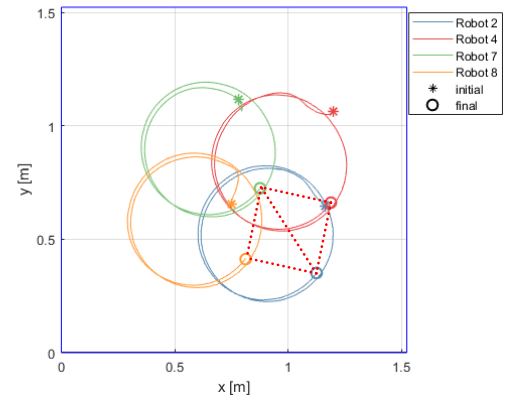
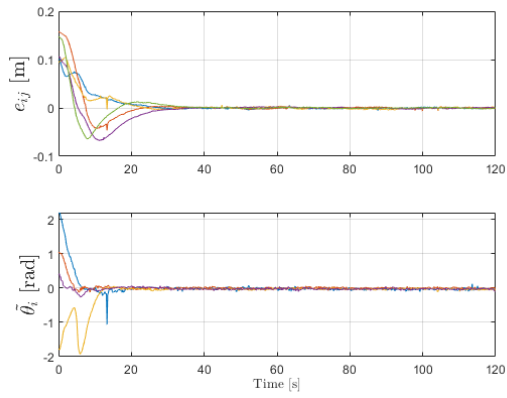


i) Translation from initial conditions to desired formation (left to right)

Mode 1



Mode 2



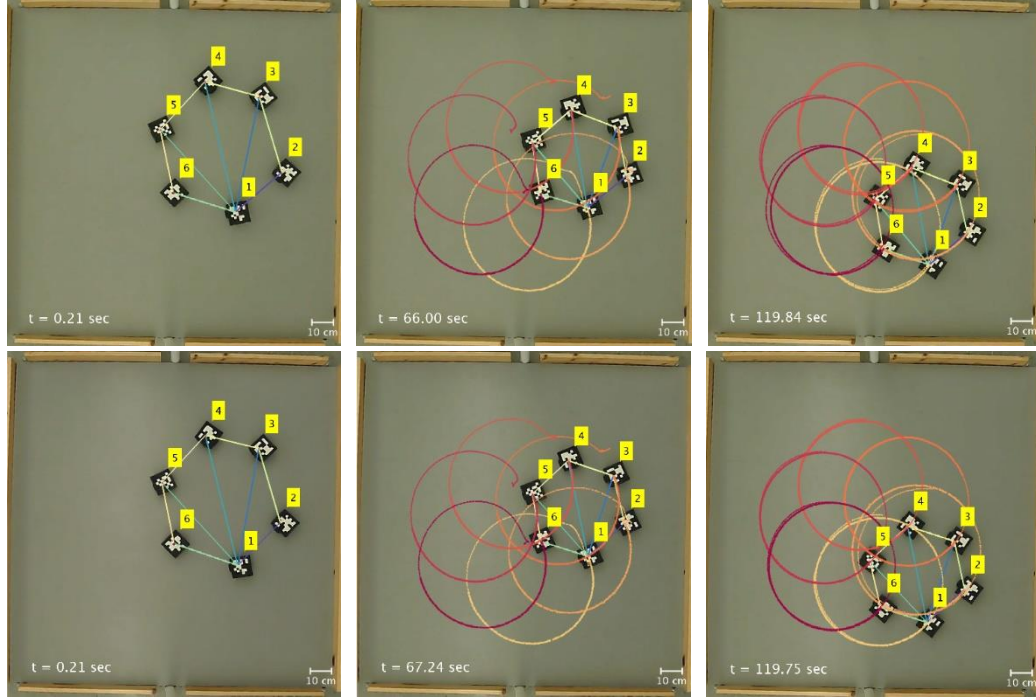
ii) Distance and heading angle errors

iii) Robot paths

Figure 4.8. Formation maneuvering with four robots.

Mode 1

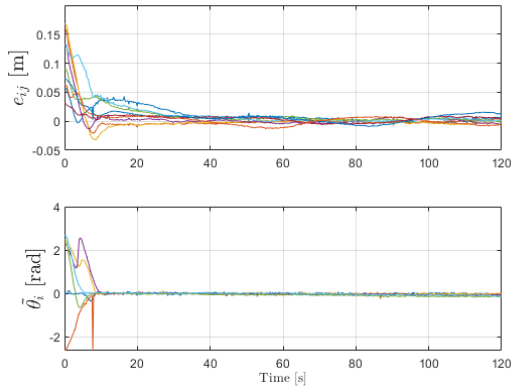
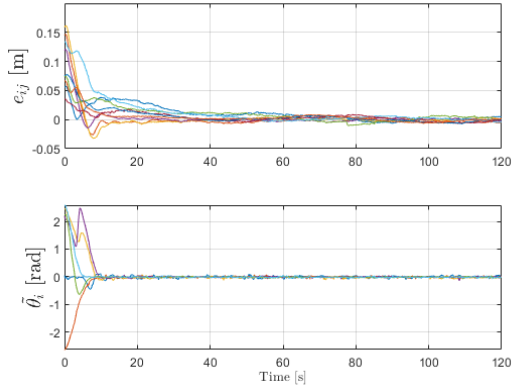
Mode 2



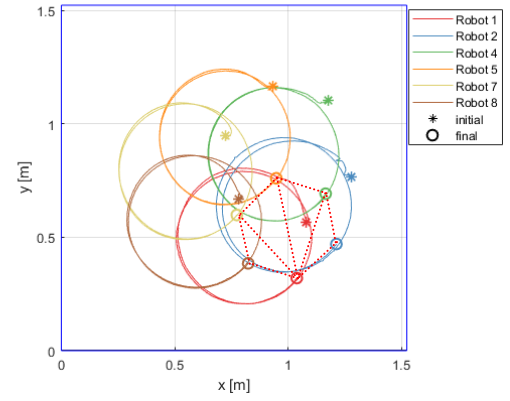
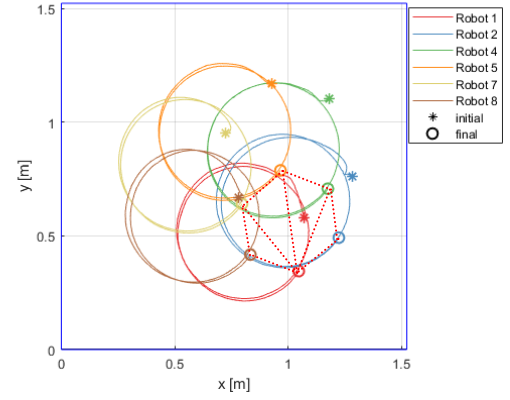
i) Translation from initial conditions to desired formation (left to right)

Mode 1

Mode 2



ii) Distance and heading angle errors

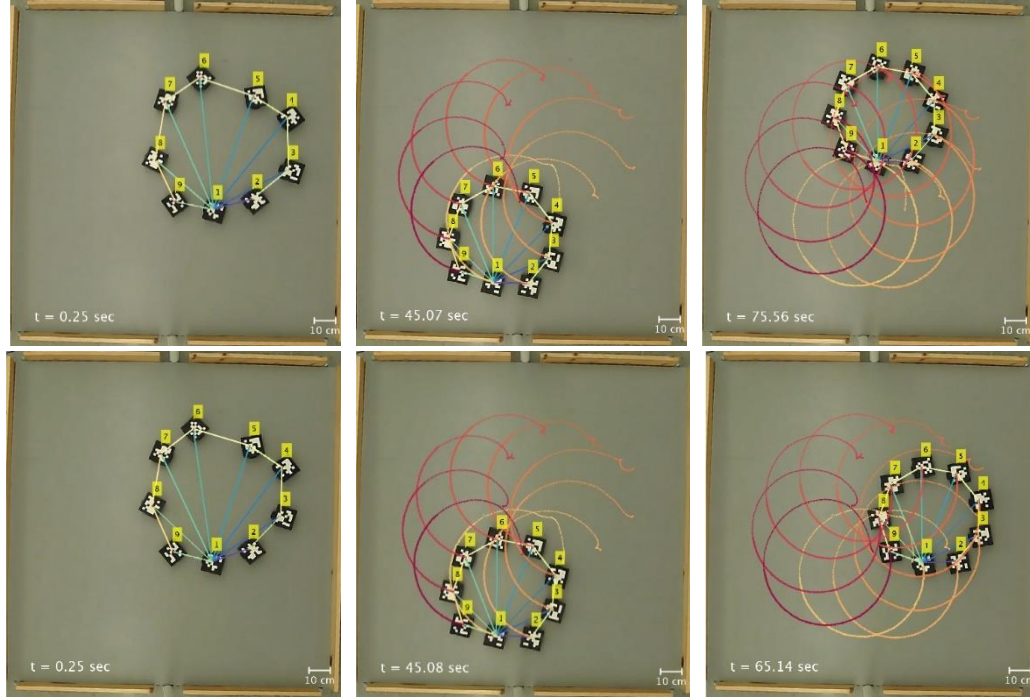


iii) Robot paths

Figure 4.9. Formation maneuvering with six robots.

Mode 1

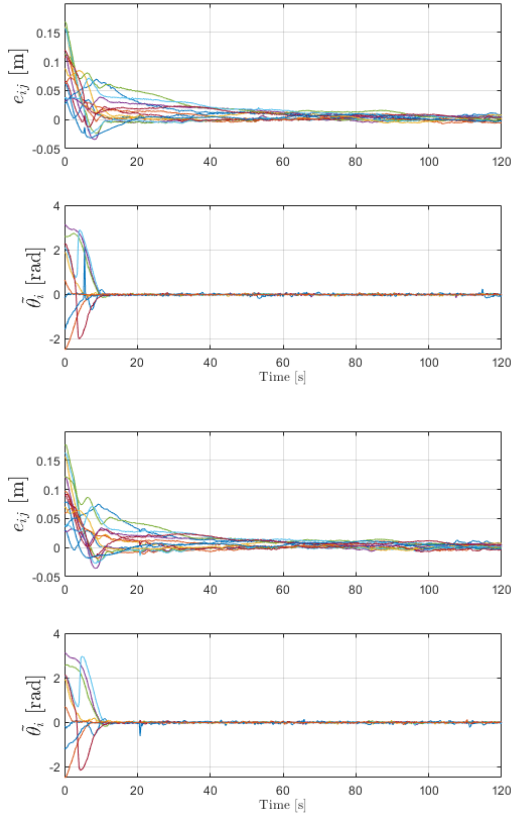
Mode 2



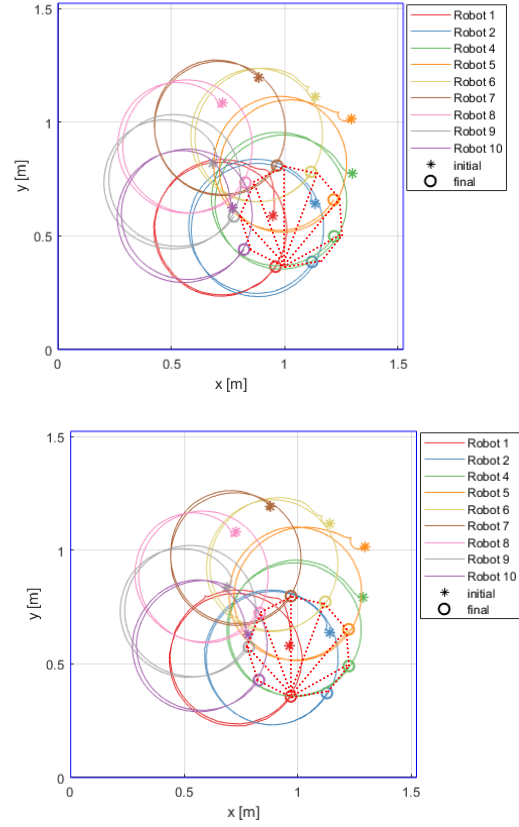
i) Translation from initial conditions to desired formation (left to right)

Mode 1

Mode 2



ii) Distance and heading angle errors



iii) Robot paths

Figure 4.10. Formation maneuvering with nine robots.

Table 4.4. Summary of Formation Maneuvering Experiments.

No. of robots	3		4		6		9	
Operating Modes	<i>Mode 1</i>	<i>Mode 2</i>	<i>Mode 1</i>	<i>Mode 2</i>	<i>Mode 1</i>	<i>Mode 2</i>	<i>Mode 1</i>	<i>Mode 2</i>
Distance error								
Convergence Time(s)	20.96	22.24	43.38	37.62	61.74	67.20	73.00	64.50
Steady-State Error range(cm)	[-0.63, 0.54]	[-0.53, 0.54]	[-1.0, 0.78]	[-0.62, 0.59]	[-1.06, 0.89]	[-0.92, 1.60]	[-0.72, 1.60]	[-0.76, 1.45]
Average Steady-State Error (cm)	0.19	0.15	0.23	0.10	0.29	0.35	0.40	0.41
Heading error								
Convergence Time(s)	13.28	12.48	23.40	22.68	11.76	10.71	13.50	15.50
Steady-State Error Range (deg)	[-8.21, 4.41]	[-8.29, 2.99]	[-6.65, 5.10]	[-6.29, 7.21]	[-8.03, 4.36]	[-13.69, 4.15]	[-9.76, 14.15]	[-12.88, 7.91]
Average Steady-State Error(deg)	1.07	1.23	1.10	1.43	1.10	2.98	1.13	1.19

Based on the robot convergence times and the average steady-state error values included in Table 4.4, one particular operating mode cannot simply be claimed to have better performance than the other. In a few cases, the convergence time in one mode was larger than the other, however the average steady-state error was smaller, and vice versa.

Even after running several trails for both formation acquisition and formation maneuvering experiments, no specific pattern was found for the controllers in error handling. Otherwise in most cases, the convergence time for the robots in *Mode 1* was slightly smaller than in *Mode 2*, however that did not hold true for all the cases. In general, the robot convergence time in *Mode 3* is likely to be the largest out of all three operating modes whatever the case is. The only explanation to such behavior of the controllers in different operating modes is that the formation performance depends on how the communication subsystems do their job. Generally, considering all

experimental trials, it is safe to say that the decentralized controllers in *Mode 2* can perform almost as good as the controllers in *Mode 1*, if not better.

Chapter 5. Conclusions and Future Work

5.1. Conclusions

In order to extend the TIGER Square testbed's range of operation from centralized mode to a fully decentralized mode, each of the major control entities e.g. the sensing, the communication, and the processing of control algorithms must be modified to work in a decentralized manner. To contribute to that goal, this work specifically focused at decentralizing the processing of the distance-based controllers with TIGERBots' onboard hardware components. This project also introduced a fully functioning decentralized communication method between the robots. Both features were tested thoroughly, and experimental results were presented in the respective chapters. To recap, the TIGER Square testbed and its various subsystems were described in Chapter 2. In this chapter we also introduced the four different operating modes of the testbed that are currently being considered. Chapter 3 focused on the detail description of the TIGERBot firmware modifications for decentralized algorithm processing and decentralized communication protocol.

In chapter 4, the theoretical background of the formation control problems was covered. Next, to evaluate the performance of the control algorithms with the modified TIGER Square subsystems, a number of formation acquisition and formation maneuvering experiments were completed in different operating modes and varying number of robots. The results showed that, the robots exhibit a oscillatory, steady-state like behavior as the distance errors and the heading errors approach approximately zero. Therefore, the performance comparison in different operating modes were done based on the convergence time of the robots towards their desired formation and the average steady-state error.

Interestingly, the results from *Modes 1* and *2* did not show any clear pattern to conclude that controllers in one mode performed better than the other. Because, after several trials with the same set of experiments, the convergence time was smaller in one mode in some cases while the steady-state error was larger or vice versa with exceptions. The only formation acquisition experiment with 4 robots in *Mode 3* verified that the robots can converge to their desired formation using decentralized communication and centralized sensing information. However, the convergence time was found to be significantly higher compared to *Modes 1, 2*, while the steady-state error values in all modes were in close range to each other. Therefore, it can be assumed that the decentralized processing of the control algorithms did not cause any deterioration in the overall formation performance. The variations in the convergence times in different operating modes were likely due to the communication delays of the system.

5.2. Future Work

Integrating the decentralized processing and communication techniques in this work along with the decentralized sensing method introduced in [2], the fully decentralized mode (*Mode 4*) could be achieved for the control experiments. Next, to simulate practical scenarios during an experiment, such as loss of GPS or Wi-Fi communication, the TIGER Square subsystems could be modified to switch between centralized and decentralized modes automatically without user intervention. Alongside these two obvious developments in the control scheme, a few other points could be added in the improvement list of the testbed as described below.

Collision-avoidance Algorithm Improvement

The current collision avoidance algorithm in TIGER Square adopted from [3] exhibits inconsistency during negotiating robot movement over time, previously documented in [22]. Robots having cross paths often end up blocking each other before reaching desired co-ordinates.

Although due to speed limitation of the robots, physical damage is not likely, these inconsistencies need attention. Either adopting a well-tested open-source collision avoidance algorithm or implying machine learning techniques to analyze the patterns might help reducing the irregularities.

Infrastructure Improvement

Depending on research priority and available budget, following infrastructure improvements could be introduced:

- A reinforced overhead camera housing with precise camera adjustment features.
- A mini projector placed overhead for computer-vision-based tracking applications.
- Fixed wired or wireless (preferable) charging points for robots attached to the test arena for uninterrupted experiments.

TIGERBot Circuitry Redesign

A redefined circuit design could be introduced to the TIGERBots with the following improvement points in mind:

- Fuse the robot main board and sensor board circuit to achieve a more compact board design with smaller footprint. Consider that, it risks a single point circuit failure.
- Add low-energy LED arrays to visually differentiate the operating modes, and mini-display to show significant output values during experiments.
- Get rid of the unused sensors for less power consumption and add a few more physical switches for better user controllability over the components.
- Modify the charging circuit for fixed port or wireless charging.

If image processing applications are intended, compatible camera modules could be equipped to individual robot. However, real-time image processing would likely require

significantly high processing power. Nevertheless, the NodeMCU chip proved to be decently fast regarding processing. Therefore, processing of centralized sensing data could be attempted on the robot main board by sending the camera feedback directly to the robots wirelessly. To achieve this, following upgrades could be considered:

- Replace the current NodeMCU microcontroller in the main board with a small size microprocessor (for example, Raspberry Pi Zero W), which offers the same key features as NodeMCU, but have more processing power and capable of parallel tasking.
- A second NodeMCU could be equipped to the robot main board for supplementary tasks (e.g., image processing, data updates in clouds etc.)

Either way, the robot circuitry must be reconfigured, and battery capacity needs to be increased.

Wireless Debugging Display

As of now, the only way of debugging the robot firmware code and checking output results is printing values on the serial monitor that comes with the IDE (Arduino IDE is used for programming the robot firmware). As the serial monitor does not work wirelessly, a USB cable always needs to be connected with the robot during debugging. This is quite a nuisance for a moving robot, even more so for multiple moving robots. There are a few third-party wireless serial-monitor libraries available that could be introduced to the firmware. However, they only allow you to see outputs from one single microcontroller at a time.

References

- [1] A. Jimenez-Gonzalez, J. R. M.-d. Dios, and A. Ollero, "Testbeds for ubiquitous robotics: A survey," *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1487-1501, 2013.
- [2] A. Kohlbacher, J. Eliasson, K. Acres, H. Chung, and J. C. Barca, "A low cost omnidirectional relative localization sensor for swarm applications," *IEEE 4th World Forum on Internet of Things (WF-IoT)*, pp. 694-699, Singapore, 2018.
- [3] D. Pickem, P. Glotfelter, L. Wang, M. Mote, A. Ames, E. Feron, and M. Egerstedt, "The Robotarium: A remotely accessible swarm robotics research testbed," in *Proceedings IEEE International Conference on Robotics and Automation*, Singapore, 2017.
- [4] E. Guizzo and E. Ackerman, "The TurtleBot3 Teacher [Resources_Hands On]," in *IEEE Spectrum*, vol. 54, no. 8, pp. 19-20, 2017.
- [5] "ESP-NOW - ESP32 - ESP-IDF Programming Guide latest documentation". Accessed on: June 27, 2021. [online]. Available at: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
- [6] F. Arvin, F., M. Espinosa, L. Jose, B. Bird, A. West, S. Watson, and B. Lennox, "Mona: an Affordable Open-Source Mobile Robot for Education and Research," in *Journal of Intelligent & Robotic Systems*, vol. 94, pp. 761-775, 2019.
- [7] F. Rivard, J. Bisson, F. Michaud, and L. Dominic, "Ultrasonic Relative Positioning for Multi-Robot Systems," in *IEEE International Conference on Robotics and Automation*, pp. 323-328, Pasadena, CA, 2008.
- [8] G. Campion, G. Bastin, and B. D'Andrea-Novet, "Structural properties and classification of kinematic and dynamic models of wheeled mobile robots," *IEEE Transactions on Robotics and Automations*, vol. 12, no.1, pp. 47-62, 1996.
- [9] J. M. Soares, I. Navarro, and A. Martinoli, "The Khepera IV Mobile Robot: Performance Evaluation, Sensory Data and Software Toolbox," *ROBOT 2015: Second Iberian Robotics Conference*, Lisbon, Portugal, pp.767-781, 2015.
- [10] J. A. Tran, P. Ghosh, Y. Gu, R. Kim, D. D'Souza, N. Ayanian, and B. Krishnamachari, "Intelligent robotic IoT system (IRIS) testbed," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [11] J. F. Roberts, T. S. Stirling, J. C. Zufferey, and D. Floreano, "2.5D infrared range and bearing system for collective robotics," *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2009*, pp. 3659-3664, 2009.

- [12] J. F. Roberts, T. Stirling, J. C. Zeffrey, and D. Floreano, "3-D relative positioning sensor for indoor flying robots," *Autonomous Robots*, vol. 33, no. 1-2, pp. 5-20, 2012.
- [13] J. Pugh, X. Raemy, C. Favre, R. Falconi, and A. Martinoli, "A fast onboard relative positioning module for multirobot systems," *IEEE/ASME Transactions on Mechatronics*, vol. 14, no. 2, pp. 151-162, 2009.
- [14] J. Wang and E. Olson, "AprilTag 2: Efficient and robust fiducial detection," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4193-4198, 2016.
- [15] L. Mao, J. Chen, Z. Li, and D. Zhang, "Relative localization method of multiple micro robots based on simple sensors," *International Journal of Advanced Robotic Systems*, vol. 10, pp. 1-9, 2013.
- [16] M. de Queiroz, X. Cai, and M. Feemster, *Formation Control of Multi-Agent Systems: A Graph Rigidity Approach*. Hoboken, NJ: Wiley, 2019.
- [17] "Overview - NodeMCU Documentation". Accessed on: June 27, 2021. [online]. Available at: <https://nodemcu.readthedocs.io/en/release/#nodemcu-documentation>
- [18] R. Michael, C. Ahler, and R. Nagpal, "Kilobot: A Low Cost Scalable Robot System for Collective Behaviors," in Proceedings *IEEE International Conference on Robotics and Automation*, Saint Paul, Minnesota, pp. 3293-3298, 2012.
- [19] S. Williams, *Distance-Based Formation Control using Decentralized Sensing with Infrared Photodiodes*. M.S. Thesis, 2021, Louisiana State University, Baton Rouge, LA.
- [20] T. Liu, M. de Queiroz, P. Zhang, and M. Khaledyan, "Further Results on Distance and Area Control of Planar Formations," *International Journal of Control*, vol. 94, no. 3, pp. 767-783, 2021.
- [21] T. Sarker, M. de Queiroz, "Towards Decentralized Experimental Implementation of Distance-based Formation Controllers," Poster presentation, *IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 2021.
- [22] V. Fernandez-Kim, *A Low-Cost Experimental Test Bed for Multi-Agent System Coordination Control*. M.S. Thesis, 2019, Louisiana State University, Baton Rouge, LA.
- [23] X. Cai and M. de Queiroz, "Adaptive Rigidity-Based Formation Control for Multi-Robotic Vehicles with Dynamics," *IEEE Transactions on Control Systems Technology*, vol. 23, no. 3-4, pp. 79-84, 2015.
- [24] Y. Tan and Z. yang Zheng, "Research Advance in Swarm Robotics," *Defence Technology*, vol. 9, no. 1, pp. 18-39, 2013.

Vita

Tonmoy Sarker is originally from Dhaka, Bangladesh. He went to Notre Dame College, Dhaka and got his higher secondary degree (H.S.C.) there in 2012. He attended the Military Institute of Science and Technology in Dhaka where he earned his Bachelor of Science (B.Sc.) degree in Aeronautical Engineering (Major: Avionics) in 2017. He enrolled in the Mechanical Engineering graduate program in Louisiana State University, Baton Rouge in Fall, 2018. He plans to receive his master's degree in Mechanical Engineering (minor in Electrical Engineering) in August 2021. After graduation, he intends to continue his Ph.D. degree in the fields of robotics.